В этом домашнем задании вам предстоит самостоятельно решить задачу классификации текстов на основе семинарского кода. Мы будем использовать датасет ag_news. Это датасет для классификации новостей на 4 темы: "World", "Sports", "Business", "Sci/Tech".

Установим модуль datasets, чтобы нам проще было работать с данными.

```
# import os
# os.environ['CUDA_LAUNCH_BLOCKING'] = '1'

# !pip install datasets
```

Импорт необходимых библиотек

```
import os
import random
import numpy as np
import torch
import sys

# Фиксирует все источники случайности
def set_seed(seed=42):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)

    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False
```

```
    os.environ['PYTHONHASHSEED'] = str(seed)

set_seed(42)

import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
import datasets

import matplotlib.pyplot as plt

from tqdm.auto import tqdm
# from datasets import load_dataset
from nltk.tokenize import word_tokenize
from sklearn.model_selection import train_test_split
import nltk

from collections import Counter
from typing import List
import string

import seaborn
seaborn.set_theme(palette='summer')

# nltk.download('punkt')

device = 'cuda' if torch.cuda.is_available() else 'cpu'
device
```

```
'cuda'
```

Подготовка данных

Для вашего удобства, мы привели код обработки датасета в ноутбуке. Ваша задача --- обучить модель, которая получит максимальное возможное качество на тестовой части.

```
# Загрузим датасет
dataset = datasets.load_dataset('ag_news')
```

```
Warning: You are sending unauthenticated requests to the HF Hub.
Please set a HF_TOKEN to enable higher rate limits and faster
downloads.
```

Как и в семинаре, выполним следующие шаги:

- Составим словарь
- Создадим класс WordDataset
- Выделим обучающую и тестовую часть, создадим DataLoader-ы.

```
words = Counter()

for example in tqdm(dataset['train']['text']):
    # Приводим к нижнему регистру и убираем пунктуацию
    prccessed_text = example.lower().translate(
        str.maketrans('', '', string.punctuation))

    for word in word_tokenize(prccessed_text):
        words[word] += 1


vocab = set(['<unk>', '<bos>', '<eos>', '<pad>'])
counter_threshold = 25

for char, cnt in words.items():
    if cnt > counter_threshold:
        vocab.add(char)

print(f'Размер словаря: {len(vocab)}')

word2ind = {char: i for i, char in enumerate(vocab)}
ind2word = {i: char for char, i in word2ind.items()}
```

{"model_id":"149657ee5a78415eae8b196ee709e0a2","version_major":2,"version_minor":0}

```
Размер словаря: 11842
```

```
class WordDataset:
    def __init__(self, sentences):
        self.data = sentences
        self.unk_id = word2ind['<unk>']
        self.bos_id = word2ind['<bos>']
        self.eos_id = word2ind['<eos>']
        self.pad_id = word2ind['<pad>']

    def __getitem__(self, idx: int) -> List[int]:
        processed_text = self.data[idx]['text'].lower().translate(
            str.maketrans('', '', string.punctuation))
        tokenized_sentence = [self.bos_id]
        tokenized_sentence += [
            word2ind.get(word, self.unk_id) for word in
word_tokenize(processed_text)
            ]
        tokenized_sentence += [self.eos_id]

        train_sample = {
            'text': tokenized_sentence,
            'label': self.data[idx]['label']
        }
```

```
        return train_sample

    def __len__(self) -> int:
        return len(self.data)


def collate_fn_with_padding(
    input_batch: List[List[int]], pad_id=word2ind['<pad>'],
max_len=256) -> torch.Tensor:
    seq_lens = [len(x['text']) for x in input_batch]
    max_seq_len = min(max(seq_lens), max_len)

    new_batch = []
    for sequence in input_batch:
        sequence['text'] = sequence['text'][:max_seq_len]
        for _ in range(max_seq_len - len(sequence['text'])):
            sequence['text'].append(pad_id)

        new_batch.append(sequence['text'])

    sequences = torch.LongTensor(new_batch).to(device)
    labels = torch.LongTensor([x['label'] for x in
input_batch]).to(device)

    new_batch = {
        'input_ids': sequences,
        'label': labels
    }

    return new_batch

train_dataset = WordDataset(dataset['train'])

np.random.seed(42)
idx = np.random.choice(np.arange(len(dataset['test'])), 5000)
eval_dataset = WordDataset(dataset['test'].select(idx))

batch_size = 32
train_dataloader = DataLoader(
    train_dataset, shuffle=True, collate_fn=collate_fn_with_padding,
batch_size=batch_size)

eval_dataloader = DataLoader(
    eval_dataset, shuffle=False, collate_fn=collate_fn_with_padding,
batch_size=batch_size)
```

Постановка задачи

Ваша задача -- получить максимальное возможное accuracy на `eval_dataloader`. Ниже приведена функция, которую вам необходимо запустить для обученной модели, чтобы вычислить качество её работы.

```
def evaluate(model, eval_dataloader) -> float:
    """
    Calculate accuracy on validation dataloader.
    """

    predictions = []
    target = []
    with torch.no_grad():
        for batch in eval_dataloader:
            logits = model(batch['input_ids'])
            predictions.append(logits.argmax(dim=1))
            target.append(batch['label'])

    predictions = torch.cat(predictions)
    target = torch.cat(target)
    accuracy = (predictions == target).float().mean().item()

    return accuracy
```

Ход работы

Оценка за домашнее задание складывается из четырех частей:

Запуск базовой модели с семинара на новом датасете (1 балл)

На семинаре мы создали модель, которая дает на нашей задаче довольно высокое качество. Ваша цель --- обучить ее и вычислить `score`, который затем можно будет использовать в качестве бейзлайна.

В модели появится одно важное изменение: количество классов теперь равно не 2, а 4. Обратите на это внимание и найдите, что в коде создания модели нужно модифицировать, чтобы учесть это различие.

Base model

```
class CharLM(nn.Module):
    def __init__(
        self, hidden_dim: int,
        vocab_size: int,
        num_classes: int = 4,   # 4 класса
        aggregation_type: str = 'max'
        ):
        super().__init__()
```

```
        self.embedding = nn.Embedding(vocab_size, hidden_dim)
        self.rnn = nn.RNN(hidden_dim, hidden_dim, batch_first=True)
        self.linear = nn.Linear(hidden_dim, hidden_dim)
        self.projection = nn.Linear(hidden_dim, num_classes)

        self.non_lin = nn.Tanh()
        self.dropout = nn.Dropout(p=0.1)

        self.aggregation_type = aggregation_type

    def forward(self, input_batch) -> torch.Tensor:
        embeddings = self.embedding(input_batch)  # [batch_size,
seq_len, hidden_dim]
        output, _ = self.rnn(embeddings)  # [batch_size, seq_len,
hidden_dim]

        if self.aggregation_type == 'max':
            output = output.max(dim=1)[0] #[batch_size, hidden_dim]
        elif self.aggregation_type == 'mean':
            output = output.mean(dim=1) #[batch_size, hidden_dim]
        else:
            raise ValueError('Invalid aggregation_type')

        output = self.dropout(self.linear(self.non_lin(output)))  #
[batch_size, hidden_dim]
        prediction = self.projection(self.non_lin(output))  #
[batch_size, num_classes]

        return prediction

model = CharLM(hidden_dim=256, vocab_size=len(vocab)).to(device)
criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
optimizer = torch.optim.Adam(model.parameters())

num_epoch = 5
eval_steps = len(train_dataloader) // 2

losses_type = {}
acc_type = {}

for aggregation_type in ['max', 'mean']:
    print(f'Starting training for {aggregation_type}')
    losses = []
    acc = []

    model = CharLM(hidden_dim=256, vocab_size=len(vocab),
aggregation_type=aggregation_type).to(device)
    criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
    optimizer = torch.optim.Adam(model.parameters())

    for epoch in range(num_epoch):
```

```
        epoch_losses = []
        model.train()
        for i, batch in enumerate(tqdm(train_dataloader,
desc=f'Training epoch {epoch}:')):
            optimizer.zero_grad()
            logits = model(batch['input_ids'])
            loss = criterion(logits, batch['label'])
            loss.backward()
            optimizer.step()

            epoch_losses.append(loss.item())
            if i % eval_steps == 0:
                model.eval()
                acc.append(evaluate(model, eval_dataloader)) # +
eval_dataloader
                model.train()

        losses.append(sum(epoch_losses) / len(epoch_losses))

    losses_type[aggregation_type] = losses
    acc_type[aggregation_type] = acc
```

```
Starting training for max
```

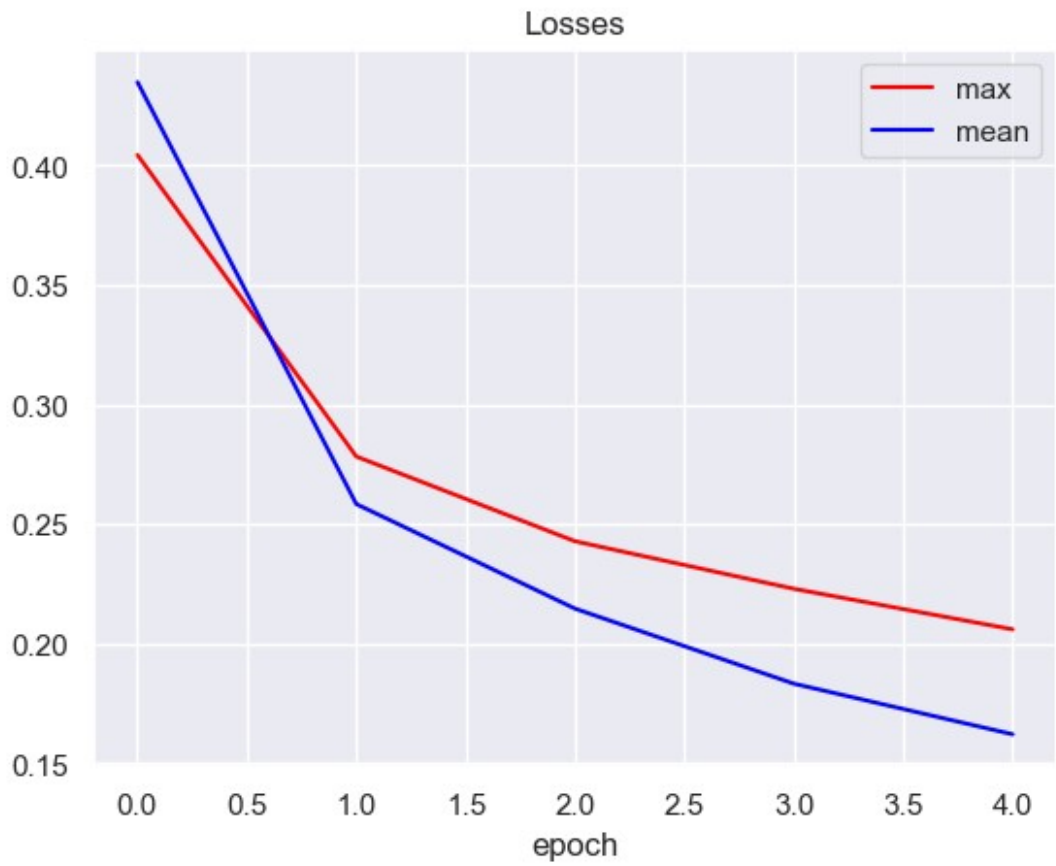

```
{"model_id":"c1482fa237d549efa266331eeda60bff","version_major":2,"vers
ion_minor":0}

{"model_id":"aec9c4f3037b47519ca7ae42579ab2d8","version_major":2,"vers
ion_minor":0}

{"model_id":"afab9df4b3464822b10386febd7cf43a","version_major":2,"vers
ion_minor":0}

{"model_id":"e5bb8fce729e4580af68301b5aa128ba","version_major":2,"vers
ion_minor":0}

{"model_id":"e4ae35ba321948cc9e15c7abd7ff1895","version_major":2,"vers
ion_minor":0}
```


```
Starting training for mean
```


```
{"model_id":"46b0e9d7e88e442aba176a2765a1fdc8","version_major":2,"vers
ion_minor":0}

{"model_id":"10e2d1959dd74d34ba989d6939eea464","version_major":2,"vers
ion_minor":0}

{"model_id":"f9331458b3e14971982d3411ead30667","version_major":2,"vers
ion_minor":0}

{"model_id":"fe103e40aba147068b15ec0db1197109","version_major":2,"vers
ion_minor":0}
```

{"model_id":"e44859baf76041409a92d44dac83fa54","version_major":2,"version_minor":0}

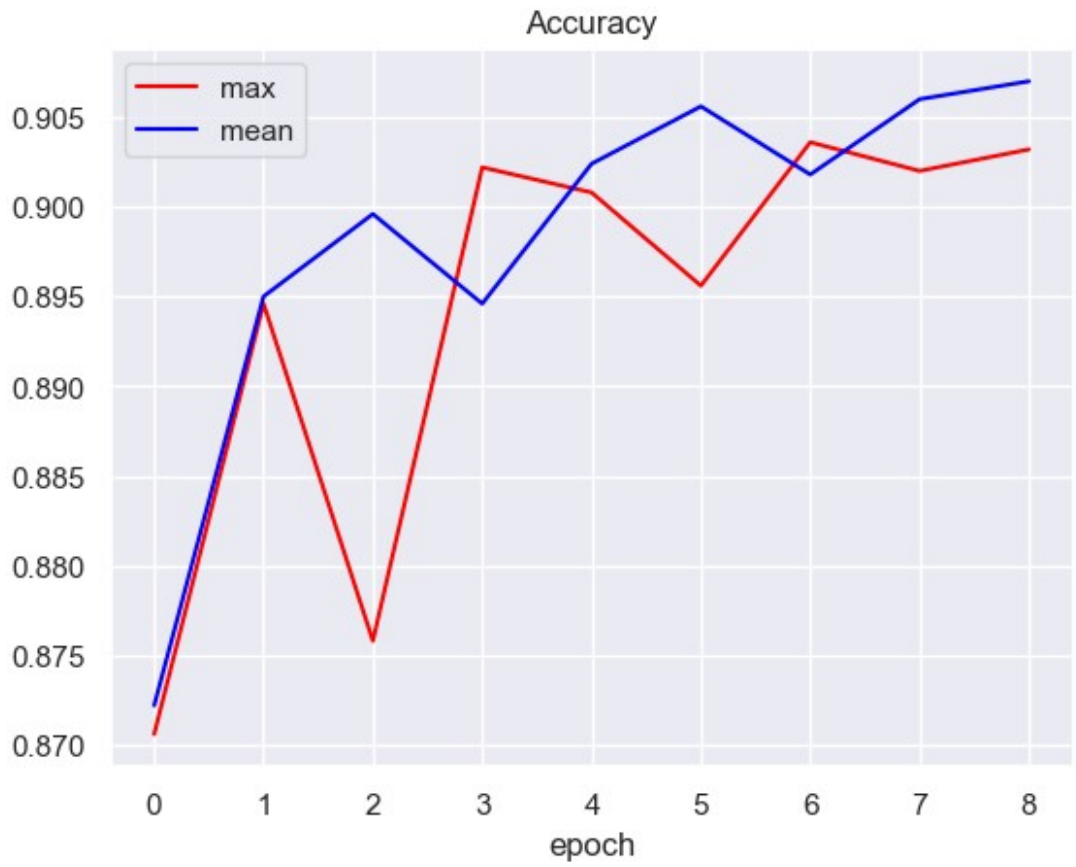
```
for (name, values), color in zip(losses_type.items(), ['red',
'blue']):
    plt.plot(np.arange(len(losses_type[name])), losses_type[name],
color=color, label=name)

plt.title('Losses')
plt.xlabel('epoch')
plt.legend()

plt.show();
```




```
for (name, values), color in zip(losses_type.items(), ['red',
'blue']):
    plt.plot(np.arange(len(acc_type[name][1:])), acc_type[name][1:],
color=color, label=name)
    print(f'Лучшая accuracy для подхода {name}: {(max(acc_type[name])
* 100):.2f}')

plt.title('Accuracy')
plt.xlabel('epoch')
```

```
plt.legend()

plt.show();
```

```
Лучшая accuracy для подхода max: 90.36
Лучшая accuracy для подхода mean: 90.70
```



Проведение экспериментов по улучшению модели (2 балла за каждый эксперимент)

Чтобы улучшить качество базовой модели, можно попробовать различные идеи экспериментов. Каждый выполненный эксперимент будет оцениваться в 2 балла. Для получения полного балла за этот пункт вам необходимо выполнить по крайней мере 2 эксперимента. Не расстраивайтесь, если какой-то эксперимент не дал вам прироста к качеству: он все равно зачтется, если выполнен корректно.

Вот несколько идей экспериментов:

- **Модель RNN**. Попробуйте другие нейросетевые модели --- LSTM и GRU. Мы советуем обратить внимание на GRU, так как интерфейс этого класса ничем не отличается от обычной Vanilla RNN, которую мы использовали на семинаре.

- **Увеличение количества рекуррентных слоев модели**. Это можно сделать с помощью параметра `num_layers` в классе `nn.RNN`. В такой модели выходы первой RNN передаются в качестве входов второй RNN и так далее.
- **Изменение архитектуры после применения RNN**. В базовой модели используется агрегация со всех эмбеддингов. Возможно, вы захотите конкатенировать результат агрегации и эмбеддинг с последнего токена.
- **Подбор гиперпараметров и обучение до сходимости**. Возможно, для получения более высокого качества просто необходимо увеличить количество эпох обучения нейросети, а также попробовать различные гиперпараметры: размер словаря, `dropout_rate`, `hidden_dim`.

Обратите внимание, что главное правило проведения экспериментов --- необходимо совершать одно архитектурное изменение в одном эксперименте. Если вы совершите несколько изменений, то будет неясно, какое именно из изменений дало прирост к качеству.

Таблица для сравнения моделей

```
import pandas as pd

# Значения Loss и Accuracy базовой модели RNN
base_max_loss = losses_type['max']
base_max_acc = acc_type['max']

base_mean_loss = losses_type['mean']
base_mean_acc = acc_type['mean']

# Сбор статистик для моделей
df_stats = pd.DataFrame(columns=['Model', 'Min_loss', 'Max_accuracy',
'Diff_accuracy'])

loss_base_max = min(base_max_loss)
acc_base_max = max(base_max_acc)

loss_base_mean = min(base_mean_loss)
acc_base_mean = max(base_mean_acc)

data_to_add = [
    ('base_mean', loss_base_mean, acc_base_mean, acc_base_mean -
acc_base_mean),
    ('base_max', loss_base_max, acc_base_max, acc_base_max -
acc_base_mean)
]

for i, (model, min_loss, max_accuracy, diff_accuracy) in
enumerate(data_to_add):
    df_stats.loc[i] = [model, min_loss, max_accuracy, diff_accuracy]

display(df_stats)
```

```
       Model  Min_loss  Max_accuracy  Diff_accuracy
0  base_mean  0.162125        0.9070         0.0000
1   base_max  0.206021        0.9036        -0.0034
```

Для обучения и сравнения экспериментальной модели с базовой моделью RNN (mean подход) напишем функцию. При агрегации max в моделях сравниваем модели с base моделью с агрегацией max, в остальных случаях сравниваем с лучшей метрикой accuracy в base модели - mean.

```
from IPython.display import clear_output

# Функция для тренировки модели и сравнения текущей модели с базовой
моделью RNN
def train_model(model, criterion, optimizer,
                model_name='current',
                aggregation_type='mean',
                plot=True,
                num_epoch=5,
                base_mean_loss=base_mean_loss,
                base_mean_acc=base_mean_acc,
                train_loader=train_dataloader,
                test_loader=eval_dataloader):

    eval_steps = len(train_loader) // 2

    total_loss = []
    total_acc = []

    print(f'\n{model_name} модель')

    for epoch in tqdm(range(num_epoch), desc=f'Training
{model_name}'):
        epoch_train_loss = 0

        model.train()
        for i, batch in enumerate(train_loader):
            optimizer.zero_grad()
            logits = model(batch['input_ids'])
            loss = criterion(logits, batch['label'])
            loss.backward()
            optimizer.step()
            epoch_train_loss += loss.item()

            if i % eval_steps == 0:
                model.eval()
                total_acc.append(evaluate(model, test_loader))
                model.train()

        total_loss.append(epoch_train_loss / len(train_loader))
```

```
        if aggregation_type == 'max':
            base_loss = base_max_loss
            base_acc = base_max_acc
        else:
            base_loss = base_mean_loss
            base_acc = base_mean_acc

        # Отрисовка графиков
        if plot:

            fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(15,
6))

            axes[0].plot(base_loss[:epoch+1], color='b', marker='o',
markersize=4, label='base')
            axes[0].plot(total_loss, color='r', marker='o',
markersize=4, label=model_name)

            axes[0].set_title('Losses')
            axes[0].set_xlabel('epoch')
            axes[0].set_ylabel('loss')
            axes[0].grid(True)
            axes[0].legend()


            axes[1].plot(base_acc[:(epoch+1)*2], color='b',
marker='o', markersize=4, label='base')
            axes[1].plot(total_acc, color='r', marker='o',
markersize=4, label=model_name)

            axes[1].set_title('Accuracy')
            axes[1].set_xlabel('epoch')
            axes[1].set_ylabel('accuracy')
            axes[1].set_ylim([0.87, 0.93])
            axes[1].grid(True)
            axes[1].legend()

            clear_output(wait=True)
            plt.show()

        # Наполнение таблицы
        curr_model_loss = min(total_loss)
        curr_model_acc = max(total_acc)
        diff_accuracy = curr_model_acc - max(base_acc)

        data_to_add = (model_name, curr_model_loss, curr_model_acc,
diff_accuracy)
        if model_name in df_stats['Model'].values:
            index = df_stats[df_stats['Model'] == model_name].index
        else:
```

```
            index = len(df_stats)
        df_stats.loc[index] = data_to_add

    print(f'Accuracy = {max(total_acc)}')
    print(('Хуже', 'Лучше')[curr_model_acc > acc_base_mean], 'на',
abs(diff_accuracy))
    return total_loss, total_acc
```

Base Tuned модель

Добавим несколько вариантов агрегаций (aggregation_type), возможность изменения количества слоев (num_layers) и вероятность отключения нейронов (dropout_rate)

```
class CharLM(nn.Module):
    def __init__(
        self, hidden_dim: int,
        vocab_size: int,
        num_classes: int = 4,   # 4 класса
        aggregation_type: str = 'max',
        num_layers=1,
        dropout_rate=0
        ):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, hidden_dim)

        if num_layers > 1:
            dropout = dropout_rate
        else:
            dropout = 0

        self.rnn = nn.RNN(hidden_dim, hidden_dim, batch_first=True,
num_layers=num_layers, dropout=dropout)

        if aggregation_type == 'mean_last_concat':
            agg_dim = hidden_dim * 2
        elif aggregation_type == 'multi_concat':
            agg_dim = hidden_dim * 4
        else:
            agg_dim = hidden_dim

        self.linear = nn.Linear(agg_dim, hidden_dim)
        self.projection = nn.Linear(hidden_dim, num_classes)

        self.non_lin = nn.Tanh()
        self.dropout = nn.Dropout(p=dropout if dropout else 0.1)

        self.aggregation_type = aggregation_type


    def forward(self, input_batch) -> torch.Tensor:
```

```
        embeddings = self.embedding(input_batch)  # [batch_size,
seq_len, hidden_dim]
        output, _ = self.rnn(embeddings)  # [batch_size, seq_len,
hidden_dim]
        # Max
        if self.aggregation_type == 'max':
            output = output.max(dim=1)[0] #[batch_size, hidden_dim]
        # Mean
        elif self.aggregation_type == 'mean':
            output = output.mean(dim=1) #[batch_size, hidden_dim]
        # Sum
        elif self.aggregation_type == 'sum':
            output = output.sum(dim=1)  # [batch_size, hidden_dim]
        # Max с нормализацией
        elif self.aggregation_type == 'max_norm':
            output = output.max(dim=1)[0]
            output = torch.nn.functional.normalize(output, p=2, dim=1)
# [batch_size, hidden_dim]
        # Mean с нормализацией
        elif self.aggregation_type == 'mean_norm':
            output = output.mean(dim=1)
            output = torch.nn.functional.normalize(output, p=2, dim=1)
# [batch_size, hidden_dim]
        # Mean_last_concat
        elif self.aggregation_type == 'mean_last_concat':
            mean_pool = output.mean(dim=1)
            last = output[:, -1, :]
            output = torch.cat([mean_pool, last], dim=1)  #
[batch_size, 2*hidden_dim]
        # Multi_concat
        elif self.aggregation_type == 'multi_concat':
            max_pool = output.max(dim=1)[0]
            mean_pool = output.mean(dim=1)
            first = output[:, 0, :]
            last = output[:, -1, :]
            output = torch.cat([max_pool, mean_pool, first, last],
dim=1)  # [batch_size, 4*hidden_dim]
        else:
            raise ValueError('Invalid aggregation_type')

        output = self.dropout(self.linear(self.non_lin(output)))  #
[batch_size, hidden_dim]
        prediction = self.projection(self.non_lin(output))  #
[batch_size, num_classes]

        return prediction

name_model = 'base'
```

Изменение архитектуры после применения RNN

Изменение aggregation_type

```
aggregation_types = ['max', 'mean', 'sum', 'max_norm', 'mean_norm',
'mean_last_concat', 'multi_concat']
model_names = ['base_' + agg_type for agg_type in aggregation_types]

for name_model, agg_type in zip(model_names, aggregation_types):
    model = CharLM(hidden_dim=256, vocab_size=len(vocab),
aggregation_type=agg_type).to(device)
    criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
    optimizer = torch.optim.Adam(model.parameters())

    model_loss, model_acc = train_model(model, criterion, optimizer,
model_name=name_model, plot=False, num_epoch=5)
```

base_max модель

{"model_id":"3b8ddef8040a49f3a401cfb3a3c44855","version_major":2,"version_minor":0}

```
Accuracy = 0.9041999578475952
Хуже на 0.0028000473976135254
```

base_mean модель

{"model_id":"90057b21a17941849f9b1f714cba53da","version_major":2,"version_minor":0}

```
Accuracy = 0.9081999659538269
Лучше на 0.001199960708618164
```

base_sum модель

{"model_id":"b28fb9a2575e434fbd7a447510b165e5","version_major":2,"version_minor":0}

```
Accuracy = 0.8527999520301819
Хуже на 0.054200053215026855
```

base_max_norm модель

{"model_id":"096a10e8ada24df5a209646361a0dc4a","version_major":2,"version_minor":0}

```
Accuracy = 0.9079999923706055
Лучше на 0.0009999871253967285
```

base_mean_norm модель

{"model_id":"88c33195f4d348fa8c33efdb6caf6b2e","version_major":2,"version_minor":0}

```
Accuracy = 0.9053999781608582
Хуже на 0.001600027084350586
```

base_mean_last_concat модель

{"model_id":"2a1d4a724d504ff595656a1228c53282","version_major":2,"version_minor":0}

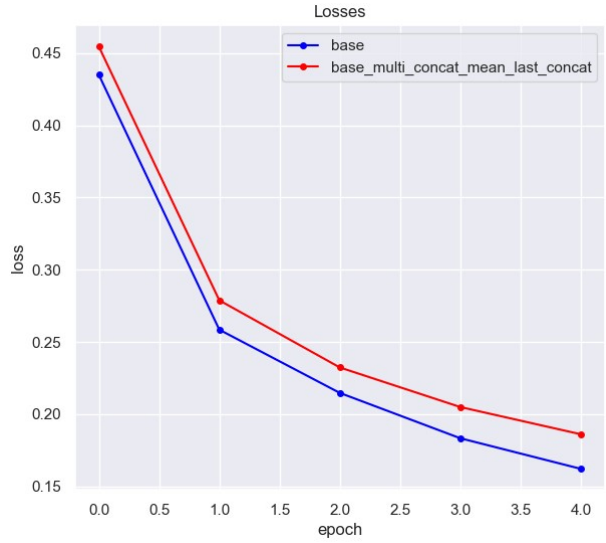
```
Accuracy = 0.8985999822616577
Хуже на 0.008400022983551025
```

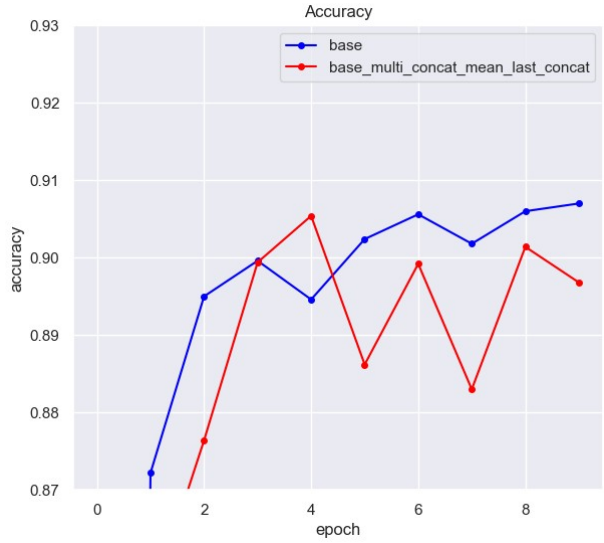
base_multi_concat модель

{"model_id":"28e5b4b7b1f448a79d724270265b542e","version_major":2,"version_minor":0}

```
Accuracy = 0.8941999673843384
Хуже на 0.012800037860870361
```

```
aggregation_types = ['sum', 'max_norm', 'mean_norm',
'mean_last_concat', 'multi_concat']

agg_type = 'mean_last_concat'

model = CharLM(hidden_dim=256, vocab_size=len(vocab),
aggregation_type=agg_type).to(device)
criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
optimizer = torch.optim.Adam(model.parameters())

model_loss, model_acc = train_model(model, criterion, optimizer,
model_name=f'{name_model}_{agg_type}')
```





```
Accuracy = 0.9053999781608582
Хуже на 0.001600027084350586

# aggregation_types = ['sum', 'max_norm', 'mean_norm',
'mean_last_concat', 'multi_concat']

# agg_type = 'sum'

# model = CharLM(hidden_dim=256, vocab_size=len(vocab),
aggregation_type=agg_type).to(device)
# criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
# optimizer = torch.optim.Adam(model.parameters())

# model_loss, model_acc = train_model(model, criterion, optimizer,
model_name=f'{name_model}_{agg_type}')

# aggregation_types = ['sum', 'max_norm', 'mean_norm',
'mean_last_concat', 'multi_concat']

# agg_type = 'max_norm'

# model = CharLM(hidden_dim=256, vocab_size=len(vocab),
aggregation_type=agg_type).to(device)
# criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
# optimizer = torch.optim.Adam(model.parameters())

# model_loss, model_acc = train_model(model, criterion, optimizer,
model_name=f'{name_model}_{agg_type}')

# aggregation_types = ['sum', 'max_norm', 'mean_norm',
'mean_last_concat', 'multi_concat']

# agg_type = 'mean_norm'

# model = CharLM(hidden_dim=256, vocab_size=len(vocab),
aggregation_type=agg_type).to(device)
# criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
# optimizer = torch.optim.Adam(model.parameters())

# model_loss, model_acc = train_model(model, criterion, optimizer,
model_name=f'{name_model}_{agg_type}')

# aggregation_types = ['sum', 'max_norm', 'mean_norm',
'mean_last_concat', 'multi_concat']

# agg_type = 'multi_concat'

# model = CharLM(hidden_dim=256, vocab_size=len(vocab),
aggregation_type=agg_type).to(device)
# criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
# optimizer = torch.optim.Adam(model.parameters())
```

```
# model_loss, model_acc = train_model(model, criterion, optimizer,
model_name=f'{name_model}_{agg_type}')
```

Увеличение количества рекуррентных слоев модели

Изменение num_layers и dropout_rate (в связке между новыми слоями)

```
import itertools

num_layers_list = [1, 2, 3]
dropout_rates = [0.0, 0.3]
agg_type = 'mean'
num_epoch = 5

for num_layers, dropout_rate in itertools.product(num_layers_list,
dropout_rates):
    model = CharLM(hidden_dim=256, vocab_size=len(vocab),
aggregation_type=agg_type, num_layers=num_layers,
dropout_rate=dropout_rate).to(device)
    criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
    optimizer = torch.optim.Adam(model.parameters())

    model_loss, model_acc = train_model(model, criterion, optimizer,
model_name=f'{name_model}_{agg_type}_nl{num_layers}_drop{dropout_rate}
',
                                        aggregation_type=agg_type,
plot=False, num_epoch=num_epoch)


base_multi_concat_mean_nl1_drop0.0 модель

{"model_id":"e1effbf43b544f33901828c4778a2a18","version_major":2,"vers
ion_minor":0}

Accuracy = 0.9089999794960022
Лучше на 0.001999974250793457

base_multi_concat_mean_nl1_drop0.3 модель

{"model_id":"382ef92be70f4631a60e2d7b9c1f9489","version_major":2,"vers
ion_minor":0}

Accuracy = 0.9059999585151672
Хуже на 0.001000046730041504

base_multi_concat_mean_nl2_drop0.0 модель

{"model_id":"92be86b37b04404bbf2d9bb87eacda4d","version_major":2,"vers
ion_minor":0}
```

```
Accuracy = 0.9107999801635742
Лучше на 0.0037999749183654785
```

base_multi_concat_mean_nl2_drop0.3 модель

{"model_id":"eb4840e745dc45d6b246a7b44b0680aa","version_major":2,"version_minor":0}

```
Accuracy = 0.9103999733924866
Лучше на 0.003399968147277832
```

base_multi_concat_mean_nl3_drop0.0 модель

{"model_id":"f5730a1f81dd40d38a27c6c6c88f8dca","version_major":2,"version_minor":0}

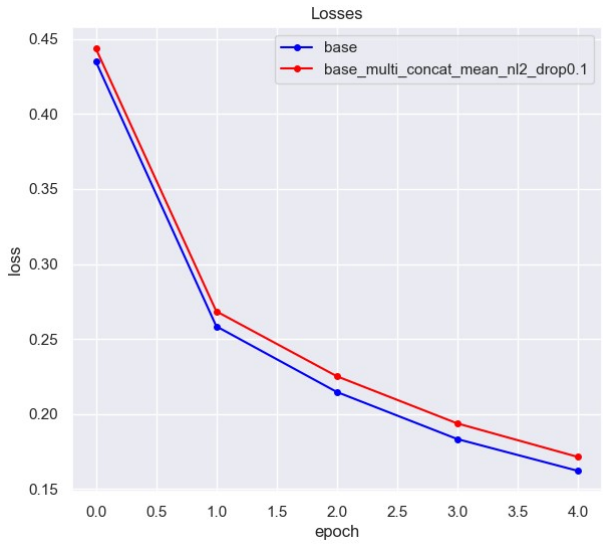
```
Accuracy = 0.9031999707221985
Хуже на 0.003800034523010254
```

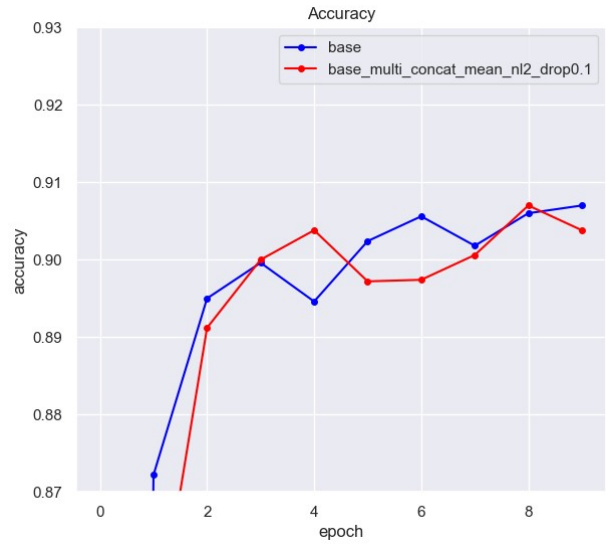
base_multi_concat_mean_nl3_drop0.3 модель

{"model_id":"32a82126e0a6457db1acdfec32b2d26f","version_major":2,"version_minor":0}

```
Accuracy = 0.9017999768257141
Хуже на 0.005200028419494629
```

```
num_layers = 2
dropout_rate = 0.1
agg_type = 'mean'

model = CharLM(hidden_dim=256, vocab_size=len(vocab),
aggregation_type=agg_type, num_layers=num_layers,
dropout_rate=dropout_rate).to(device)
criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
optimizer = torch.optim.Adam(model.parameters())

model_loss, model_acc = train_model(model, criterion, optimizer,
model_name=f'{name_model}_{agg_type}_nl{num_layers}_drop{dropout_rate}
')
```





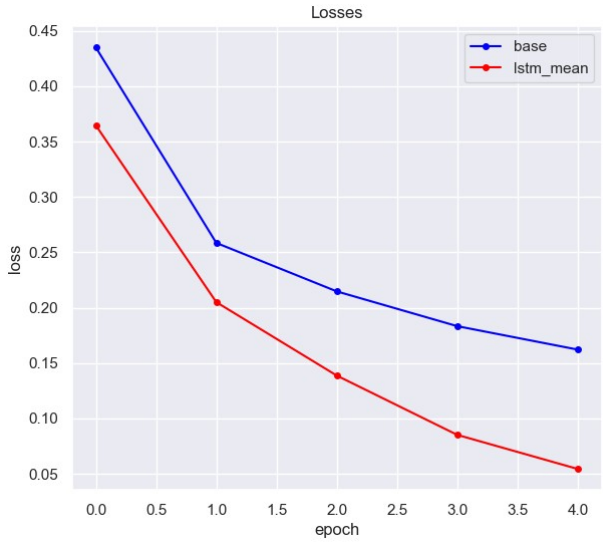

```
Accuracy = 0.9070000052452087
Хуже на 0.0

# num_layers = 2
# dropout_rate = 0
# agg_type = 'mean'

# model = CharLM(hidden_dim=256, vocab_size=len(vocab),
aggregation_type=agg_type, num_layers=num_layers,
dropout_rate=dropout_rate).to(device)
# criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
# optimizer = torch.optim.Adam(model.parameters())

# model_loss, model_acc = train_model(model, criterion, optimizer,
model_name=f'{name_model}_{agg_type}_nl{num_layers}_drop{dropout_rate}
')

# num_layers = 2
# dropout_rate = 0.2
# agg_type = 'mean'

# model = CharLM(hidden_dim=256, vocab_size=len(vocab),
aggregation_type=agg_type, num_layers=num_layers,
dropout_rate=dropout_rate).to(device)
# criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
# optimizer = torch.optim.Adam(model.parameters())

# model_loss, model_acc = train_model(model, criterion, optimizer,
model_name=f'{name_model}_{agg_type}_nl{num_layers}_drop{dropout_rate}
')

# num_layers = 3
# dropout_rate = 0
```

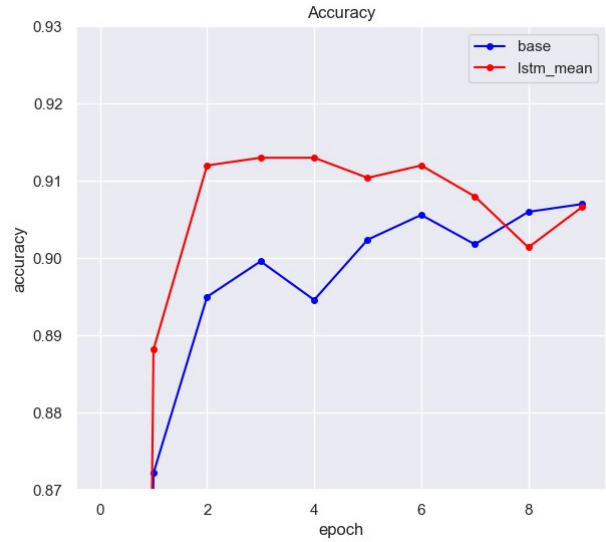
```
# agg_type = 'mean'

# model = CharLM(hidden_dim=256, vocab_size=len(vocab),
aggregation_type=agg_type, num_layers=num_layers,
dropout_rate=dropout_rate).to(device)
# criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
# optimizer = torch.optim.Adam(model.parameters())

# model_loss, model_acc = train_model(model, criterion, optimizer,
model_name=f'{name_model}_{agg_type}_nl{num_layers}_drop{dropout_rate}
')

# num_layers = 3
# dropout_rate = 0.3
# agg_type = 'mean'

# model = CharLM(hidden_dim=256, vocab_size=len(vocab),
aggregation_type=agg_type, num_layers=num_layers,
dropout_rate=dropout_rate).to(device)
# criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
# optimizer = torch.optim.Adam(model.parameters())

# model_loss, model_acc = train_model(model, criterion, optimizer,
model_name=f'{name_model}_{agg_type}_nl{num_layers}_drop{dropout_rate}
')
```

LSTM модель

```
class LSTMCharLM(nn.Module):
    def __init__(
        self, hidden_dim: int,
        vocab_size: int,
        num_classes: int = 4,
        aggregation_type: str = 'max',
        num_layers=1
        ):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, hidden_dim)
        self.rnn = nn.LSTM(hidden_dim, hidden_dim, batch_first=True,
num_layers=num_layers)
        self.linear = nn.Linear(hidden_dim, hidden_dim)
        self.projection = nn.Linear(hidden_dim, num_classes)

        self.non_lin = nn.Tanh()
        self.dropout = nn.Dropout(p=0.1)

        self.aggregation_type = aggregation_type

    def forward(self, input_batch) -> torch.Tensor:
        embeddings = self.embedding(input_batch)  # [batch_size,
```

```
seq_len, hidden_dim]
        output, _ = self.rnn(embeddings)  # [batch_size, seq_len,
hidden_dim]

        if self.aggregation_type == 'max':
            output = output.max(dim=1)[0] #[batch_size, hidden_dim]
        elif self.aggregation_type == 'mean':
            output = output.mean(dim=1) #[batch_size, hidden_dim]
        else:
            raise ValueError('Invalid aggregation_type')

        output = self.dropout(self.linear(self.non_lin(output)))  #
[batch_size, hidden_dim]
        prediction = self.projection(self.non_lin(output))  #
[batch_size, num_classes]

        return prediction

name_model = 'lstm'

agg_type = 'mean'

model = LSTMCharLM(hidden_dim=256, vocab_size=len(vocab),
aggregation_type=agg_type).to(device)
criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
optimizer = torch.optim.Adam(model.parameters())

model_loss, model_acc = train_model(model, criterion, optimizer,
model_name=f'{name_model}_{agg_type}')
```



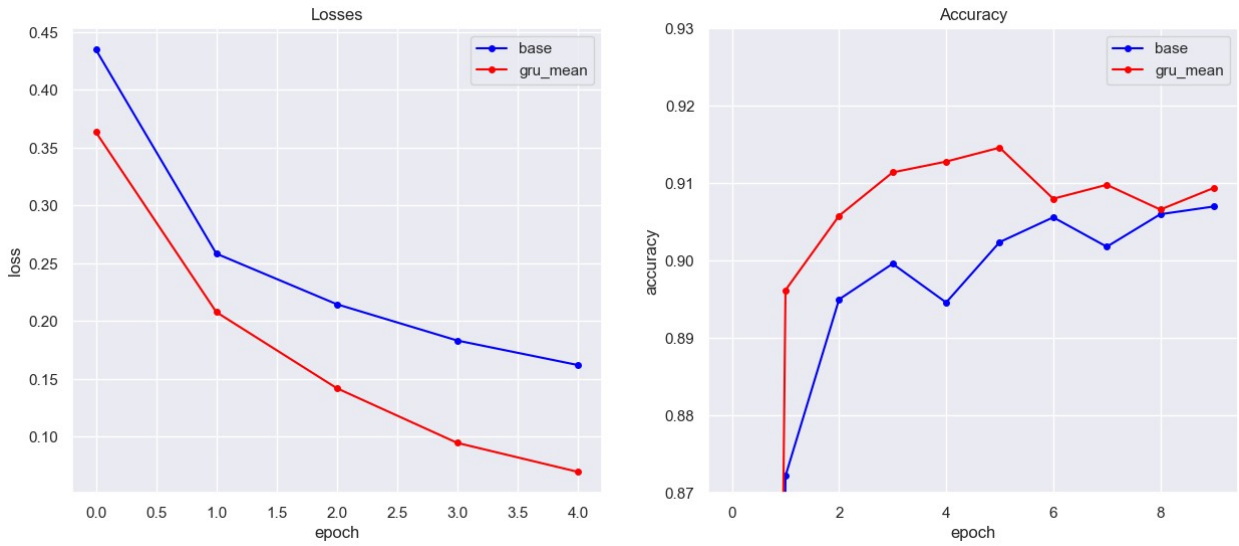



```
Accuracy = 0.9129999876022339
Лучше на 0.0059999823570251465
```

GRU модель

```
class GRUCharLM(nn.Module):
    def __init__(
        self, hidden_dim: int,
        vocab_size: int,
        num_classes: int = 4,
        aggregation_type: str = 'max',
        num_layers=1
        ):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, hidden_dim)
        self.rnn = nn.GRU(hidden_dim, hidden_dim, batch_first=True,
num_layers=num_layers)
        self.linear = nn.Linear(hidden_dim, hidden_dim)
        self.projection = nn.Linear(hidden_dim, num_classes)

        self.non_lin = nn.Tanh()
        self.dropout = nn.Dropout(p=0.1)

        self.aggregation_type = aggregation_type

    def forward(self, input_batch) -> torch.Tensor:
        embeddings = self.embedding(input_batch)  # [batch_size,
seq_len, hidden_dim]
        output, _ = self.rnn(embeddings)  # [batch_size, seq_len,
hidden_dim]

        if self.aggregation_type == 'max':
            output = output.max(dim=1)[0] #[batch_size, hidden_dim]
        elif self.aggregation_type == 'mean':
            output = output.mean(dim=1) #[batch_size, hidden_dim]
        else:
            raise ValueError('Invalid aggregation_type')

        output = self.dropout(self.linear(self.non_lin(output)))  #
[batch_size, hidden_dim]
        prediction = self.projection(self.non_lin(output))  #
[batch_size, num_classes]

        return prediction

name_model = 'gru'

agg_type = 'mean'

model = GRUCharLM(hidden_dim=256, vocab_size=len(vocab),
aggregation_type=agg_type).to(device)
criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
optimizer = torch.optim.Adam(model.parameters())
```

```
model_loss, model_acc = train_model(model, criterion, optimizer,
model_name=f'{name_model}_{agg_type}')
```

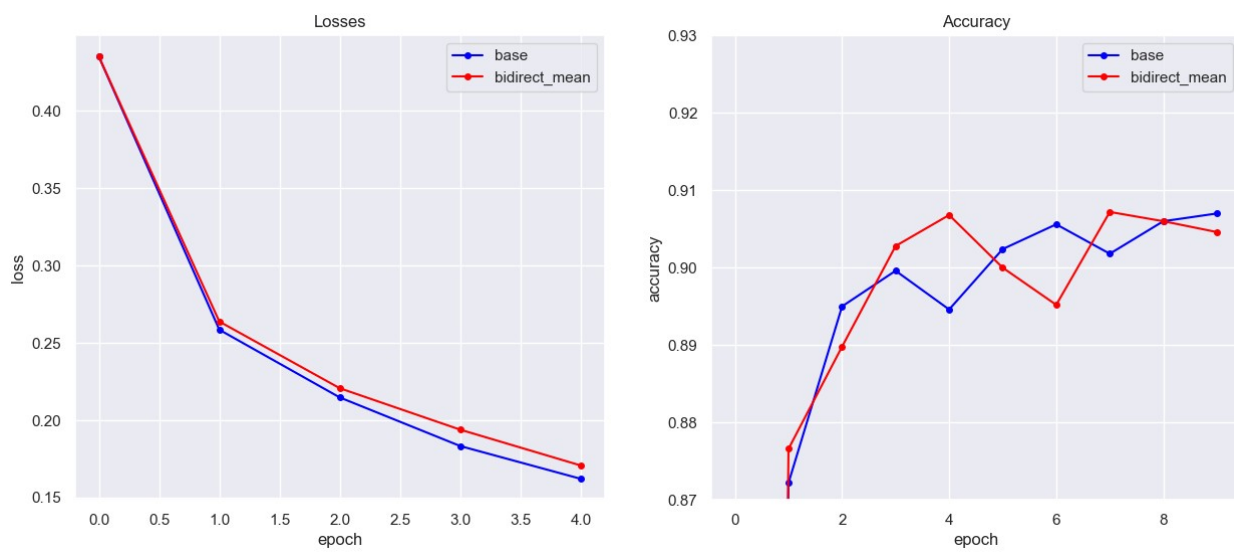



```
Accuracy = 0.9145999550819397
Лучше на 0.007599949836730957
```

Bidirectional RNN модель

```
class BidirectCharLM(nn.Module):
    def __init__(
        self, hidden_dim: int,
        vocab_size: int,
        num_classes: int = 4,
        aggregation_type: str = 'max',
        num_layers=1
        ):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, hidden_dim)
        self.rnn = nn.RNN(hidden_dim, hidden_dim, batch_first=True,
bidirectional=True, num_layers=num_layers)
        self.linear = nn.Linear(hidden_dim * 2, hidden_dim)
        self.projection = nn.Linear(hidden_dim, num_classes)

        self.non_lin = nn.Tanh()
        self.dropout = nn.Dropout(p=0.1)

        self.aggregation_type = aggregation_type

    def forward(self, input_batch) -> torch.Tensor:
        embeddings = self.embedding(input_batch)  # [batch_size,
seq_len, hidden_dim]
        output, _ = self.rnn(embeddings)  # [batch_size, seq_len,
```

```
hidden_dim]

        if self.aggregation_type == 'max':
            output = output.max(dim=1)[0] #[batch_size, hidden_dim]
        elif self.aggregation_type == 'mean':
            output = output.mean(dim=1) #[batch_size, hidden_dim]
        else:
            raise ValueError('Invalid aggregation_type')

        output = self.dropout(self.linear(self.non_lin(output)))  #
[batch_size, hidden_dim]
        prediction = self.projection(self.non_lin(output))  #
[batch_size, num_classes]

        return prediction

name_model = 'bidirect'

agg_type = 'mean'

model = BidirectCharLM(hidden_dim=256, vocab_size=len(vocab),
aggregation_type=agg_type).to(device)
criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
optimizer = torch.optim.Adam(model.parameters())

model_loss, model_acc = train_model(model, criterion, optimizer,
model_name=f'{name_model}_{agg_type}')
```

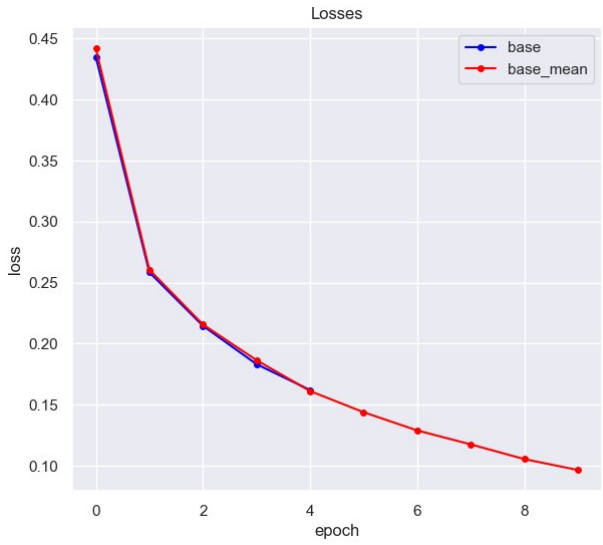


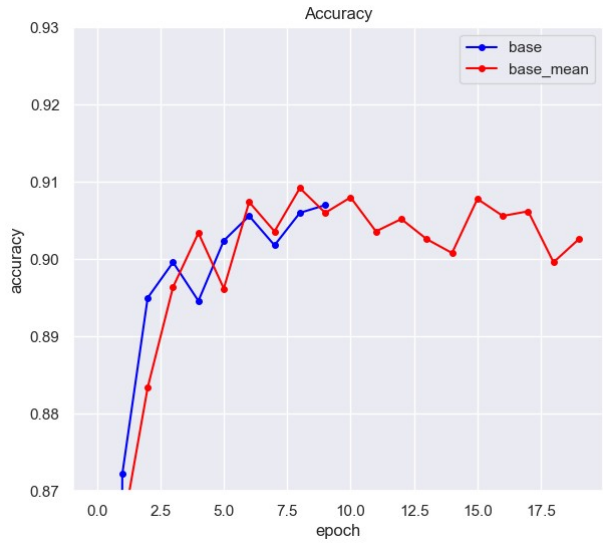

```
Accuracy = 0.9071999788284302
Лучше на 0.00019997358322143555
```

Подбор гиперпараметров и обучение до сходимости

Для чистоты эксперимента обучим базовую модель на 10 эпохах

```
num_epoch = 10

name_model = 'base'

agg_type = 'mean'

model = CharLM(hidden_dim=256, vocab_size=len(vocab),
aggregation_type=agg_type).to(device)
criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
optimizer = torch.optim.Adam(model.parameters())

base_mean_loss, base_mean_acc = train_model(model, criterion,
optimizer, model_name=f'{name_model}_{agg_type}',
                                            aggregation_type=agg_type,
plot=True, num_epoch=num_epoch, base_mean_loss=base_mean_loss,
base_mean_acc=base_mean_acc)
```






```
Accuracy = 0.9091999530792236
Лучше на 0.0021999478340148926

display(df_stats)

                                  Model  Min_loss  Max_accuracy
Diff_accuracy
0                            base_mean  0.096650        0.9092
0.0022
1                             base_max  0.207958        0.9042
-0.0028
2                             base_sum  0.530286        0.8528
```

```
-0.0542
3                        base_max_norm  0.170809        0.9080
0.0010
4                       base_mean_norm  0.169707        0.9054
-0.0016
5                base_mean_last_concat  0.186609        0.8986
-0.0084
6                    base_multi_concat  0.281257        0.8942
-0.0128
7   base_multi_concat_mean_last_concat  0.186061        0.9054
-0.0016
8   base_multi_concat_mean_nl1_drop0.0  0.163023        0.9090
0.0020
9   base_multi_concat_mean_nl1_drop0.3  0.161050        0.9060
-0.0010
10  base_multi_concat_mean_nl2_drop0.0  0.168720        0.9108
0.0038
11  base_multi_concat_mean_nl2_drop0.3  0.180615        0.9104
0.0034
12  base_multi_concat_mean_nl3_drop0.0  0.196682        0.9032
-0.0038
13  base_multi_concat_mean_nl3_drop0.3  0.226736        0.9018
-0.0052
14  base_multi_concat_mean_nl2_drop0.1  0.171408        0.9070
0.0000
15                           lstm_mean  0.054369        0.9130
0.0060
16                            gru_mean  0.069756        0.9146
0.0076
17                       bidirect_mean  0.170745        0.9072
0.0002
```

Выберем модель с наилучшим качеством при стандартных параметрах: LSTM модель

```
class LSTMCharLM(nn.Module):
    def __init__(
        self, hidden_dim: int,
        vocab_size: int,
        num_classes: int = 4,   # 4 класса
        aggregation_type: str = 'max',
        num_layers=1,
        dropout_rate=0
        ):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, hidden_dim)

        if num_layers > 1:
            dropout = dropout_rate
        else:
```

```
            dropout = 0

        self.rnn = nn.LSTM(hidden_dim, hidden_dim, batch_first=True,
num_layers=num_layers, dropout=dropout)

        if aggregation_type == 'mean_last_concat':
            agg_dim = hidden_dim * 2
        else:
            agg_dim = hidden_dim

        self.linear = nn.Linear(agg_dim, hidden_dim)
        self.projection = nn.Linear(hidden_dim, num_classes)

        self.non_lin = nn.Tanh()
        self.dropout = nn.Dropout(p=dropout if dropout else 0.1)

        self.aggregation_type = aggregation_type

    def forward(self, input_batch) -> torch.Tensor:
        embeddings = self.embedding(input_batch)  # [batch_size,
seq_len, hidden_dim]
        output, _ = self.rnn(embeddings)  # [batch_size, seq_len,
hidden_dim]
        # Max
        if self.aggregation_type == 'max':
            output = output.max(dim=1)[0] #[batch_size, hidden_dim]
        # Mean
        elif self.aggregation_type == 'mean':
            output = output.mean(dim=1) #[batch_size, hidden_dim]
        # Mean_last_concat
        elif self.aggregation_type == 'mean_last_concat':
            mean_pool = output.mean(dim=1)
            last = output[:, -1, :]
            output = torch.cat([mean_pool, last], dim=1)  #
[batch_size, 2*hidden_dim]
        else:
            raise ValueError('Invalid aggregation_type')

        output = self.dropout(self.linear(self.non_lin(output)))  #
[batch_size, hidden_dim]
        prediction = self.projection(self.non_lin(output))  #
[batch_size, num_classes]

        return prediction
```

Для этой модели реализуем перебор параметров и выберем наилучший наибор параметров

```
import itertools
```

```
name_model = 'lstm'

hidden_dims = [256, 384, 512]
aggregation_types = ['mean', 'max', 'mean_last_concat']
num_layers_list = [1, 2, 3]
dropout_rates = [0.0, 0.1, 0.2, 0.3]

num_epoch = 7 # увеличим количество эпох до 7 для лучшей сходимости

best_acc = 0
best_params = {}

for hidden_dim, num_layers, dropout_rate, agg_type in
itertools.product(hidden_dims, num_layers_list, dropout_rates,
aggregation_types):
    model = LSTMCharLM(hidden_dim=hidden_dim, vocab_size=len(vocab),
aggregation_type=agg_type, num_layers=num_layers,
dropout_rate=dropout_rate).to(device)
    criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
    optimizer = torch.optim.Adam(model.parameters())

    model_loss, model_acc = train_model(model, criterion, optimizer,
model_name=f'{name_model}_{agg_type}_hd{hidden_dim}_nl{num_layers}_dro
p{dropout_rate}',
                                        aggregation_type=agg_type,
plot=False, num_epoch=num_epoch, base_mean_loss=base_mean_loss,
base_mean_acc=base_mean_acc)

    curr_acc = max(model_acc)
    if curr_acc > best_acc:
        best_acc = curr_acc
        best_params = {
            'hidden_dim': hidden_dim,
            'aggregation_type': agg_type,
            'num_layers': num_layers,
            'dropout_rate': dropout_rate
        }
```

`lstm_mean_hd256_nl1_drop0.0 модель`

{"model_id":"d3bb2cdddfd14c5691662df7a388a470","version_major":2,"version_minor":0}

```
Accuracy = 0.917199969291687
Лучше на 0.008000016212463379
```

`lstm_max_hd256_nl1_drop0.0 модель`

{"model_id":"dc37b568c12b4b1088dcdb2606a80382","version_major":2,"version_minor":0}

```
Accuracy = 0.9085999727249146
Лучше на 0.004999995231628418

lstm_mean_last_concat_hd256_nl1_drop0.0 модель

{"model_id":"f51fb83046ae49c19edd991082f93691","version_major":2,"version_minor":0}

Accuracy = 0.9109999537467957
Лучше на 0.0018000006675720215

lstm_mean_hd256_nl1_drop0.1 модель

{"model_id":"933c77361f23424085de5bc9089d6fab","version_major":2,"version_minor":0}

Accuracy = 0.9157999753952026
Лучше на 0.006600022315979004

lstm_max_hd256_nl1_drop0.1 модель

{"model_id":"6c75898722844dcdbba8951c20edfad5","version_major":2,"version_minor":0}

Accuracy = 0.9121999740600586
Лучше на 0.008599996566772461

lstm_mean_last_concat_hd256_nl1_drop0.1 модель

{"model_id":"cd72367128cb429ca5e02ce6a80a7378","version_major":2,"version_minor":0}

Accuracy = 0.9151999950408936
Лучше на 0.006000041961669922

lstm_mean_hd256_nl1_drop0.2 модель

{"model_id":"91f33af325b248eba1e913a3f5184e76","version_major":2,"version_minor":0}

Accuracy = 0.9127999544143677
Лучше на 0.003600001335144043

lstm_max_hd256_nl1_drop0.2 модель

{"model_id":"c329741e524e4f4dae7353292a7e5510","version_major":2,"version_minor":0}

Accuracy = 0.9165999889373779
Лучше на 0.013000011444091797

lstm_mean_last_concat_hd256_nl1_drop0.2 модель
```

{"model_id":"bdec5a528ab9478c82f085b00b33446e","version_major":2,"version_minor":0}

Accuracy = 0.9145999550819397
Лучше на 0.0054000020027160645

lstm_mean_hd256_nl1_drop0.3 модель

{"model_id":"5220600480cf4d2fb519dfb3309595d9","version_major":2,"version_minor":0}

Accuracy = 0.9179999828338623
Лучше на 0.008800029754638672

lstm_max_hd256_nl1_drop0.3 модель

{"model_id":"795a864f9c8f417cada3099cd18ff7eb","version_major":2,"version_minor":0}

Accuracy = 0.9167999625205994
Лучше на 0.013199985027313232

lstm_mean_last_concat_hd256_nl1_drop0.3 модель

{"model_id":"19f2b4c15f1c44e89a05118d6936fd81","version_major":2,"version_minor":0}

Accuracy = 0.917199969291687
Лучше на 0.008000016212463379

lstm_mean_hd256_nl2_drop0.0 модель

{"model_id":"852077d47ec34d9595c6a10a89e18716","version_major":2,"version_minor":0}

Accuracy = 0.9174000024795532
Лучше на 0.00820004940032959

lstm_max_hd256_nl2_drop0.0 модель

{"model_id":"4a128ed1f74449cea8474131badb50d0","version_major":2,"version_minor":0}

Accuracy = 0.9161999821662903
Лучше на 0.01260000467300415

lstm_mean_last_concat_hd256_nl2_drop0.0 модель

{"model_id":"033909dff34a4f86aaeaa978a449e471","version_major":2,"version_minor":0}

Accuracy = 0.9169999957084656
Лучше на 0.007800042629241943

lstm_mean_hd256_nl2_drop0.1 модель

{"model_id":"2f173908b5724eaabb4c8e29d135eec0","version_major":2,"version_minor":0}

Accuracy = 0.9138000011444092
Лучше на 0.004600048065185547

lstm_max_hd256_nl2_drop0.1 модель

{"model_id":"2b7abb2d2f124c68a50a71fab22afcd4","version_major":2,"version_minor":0}

Accuracy = 0.9163999557495117
Лучше на 0.012799978256225586

lstm_mean_last_concat_hd256_nl2_drop0.1 модель

{"model_id":"7c773de1b73745719fb9b20c0e8774fd","version_major":2,"version_minor":0}

Accuracy = 0.915399968624115
Лучше на 0.006200015544891357

lstm_mean_hd256_nl2_drop0.2 модель

{"model_id":"0593efe4e0334e16851f0aec82d7df95","version_major":2,"version_minor":0}

Accuracy = 0.9169999957084656
Лучше на 0.007800042629241943

lstm_max_hd256_nl2_drop0.2 модель

{"model_id":"5ddc2b612208414abc3765f101de728a","version_major":2,"version_minor":0}

Accuracy = 0.9161999821662903
Лучше на 0.01260000467300415

lstm_mean_last_concat_hd256_nl2_drop0.2 модель

{"model_id":"acadb4182c744c7cbde52e31f6bc85b8","version_major":2,"version_minor":0}

Accuracy = 0.913599967956543
Лучше на 0.004400014877319336

lstm_mean_hd256_nl2_drop0.3 модель

{"model_id":"89463d86dd2a4e28aa953a3423bf96d8","version_major":2,"version_minor":0}

```
Accuracy = 0.9159999489784241
Лучше на 0.0067999958992004395

lstm_max_hd256_nl2_drop0.3 модель

{"model_id":"e68ada13703047b7ae865b0208224956","version_major":2,"version_minor":0}

Accuracy = 0.9169999957084656
Лучше на 0.013400018215179443

lstm_mean_last_concat_hd256_nl2_drop0.3 модель

{"model_id":"1d0bad68a8aa40868a4cbdb91263be5d","version_major":2,"version_minor":0}

Accuracy = 0.9163999557495117
Лучше на 0.007200002670288086

lstm_mean_hd256_nl3_drop0.0 модель

{"model_id":"995ec11d5fa94d0bbf0f1f09f768bbc4","version_major":2,"version_minor":0}

Accuracy = 0.9177999496459961
Лучше на 0.008599996566772461

lstm_max_hd256_nl3_drop0.0 модель

{"model_id":"87b7a6335fa6454890eca1d8ba5f8e22","version_major":2,"version_minor":0}

Accuracy = 0.9131999611854553
Лучше на 0.00959998369216919

lstm_mean_last_concat_hd256_nl3_drop0.0 модель

{"model_id":"98243d57f9c64aa6b27be20d7fe87a91","version_major":2,"version_minor":0}

Accuracy = 0.915399968624115
Лучше на 0.006200015544891357

lstm_mean_hd256_nl3_drop0.1 модель

{"model_id":"613345f14a75401c9621ce1512c88f78","version_major":2,"version_minor":0}

Accuracy = 0.9165999889373779
Лучше на 0.007400035858154297

lstm_max_hd256_nl3_drop0.1 модель
```

{"model_id":"97447f1e74544d9fb226aeb646dde48b","version_major":2,"version_minor":0}

```
Accuracy = 0.9179999828338623
Лучше на 0.014400005340576172
```

lstm_mean_last_concat_hd256_nl3_drop0.1 модель

{"model_id":"915e7760b9334c4f98c5981b7083a841","version_major":2,"version_minor":0}

```
Accuracy = 0.9147999882698059
Лучше на 0.005600035190582275
```

lstm_mean_hd256_nl3_drop0.2 модель

{"model_id":"8dae8bb9064340c2bc7b41c82581a094","version_major":2,"version_minor":0}

```
Accuracy = 0.9203999638557434
Лучше на 0.011200010776519775
```

lstm_max_hd256_nl3_drop0.2 модель

{"model_id":"de25c7170a4e4ca5a1d74bc461601641","version_major":2,"version_minor":0}

```
Accuracy = 0.9125999808311462
Лучше на 0.009000003337860107
```

lstm_mean_last_concat_hd256_nl3_drop0.2 модель

{"model_id":"f84e2ae502054f75b81887be65f51016","version_major":2,"version_minor":0}

```
Accuracy = 0.9165999889373779
Лучше на 0.007400035858154297
```

lstm_mean_hd256_nl3_drop0.3 модель

{"model_id":"4cf3989df2134bf09e5c3b88a5280aab","version_major":2,"version_minor":0}

```
Accuracy = 0.9145999550819397
Лучше на 0.005400020027160645
```

lstm_max_hd256_nl3_drop0.3 модель

{"model_id":"db5dd35a49cd4ecb9841c55787f9526d","version_major":2,"version_minor":0}

```
Accuracy = 0.911799967288971
Лучше на 0.008199989795684814
```

```
lstm_mean_last_concat_hd256_nl3_drop0.3 модель

{"model_id":"c304d382397e4e19841b4806eb009299","version_major":2,"version_minor":0}

Accuracy = 0.9143999814987183
Лучше на 0.005200028419494629

lstm_mean_hd384_nl1_drop0.0 модель

{"model_id":"78c7ac3d174b45e5a0c883c97821f17f","version_major":2,"version_minor":0}

Accuracy = 0.9175999760627747
Лучше на 0.008400022983551025

lstm_max_hd384_nl1_drop0.0 модель

{"model_id":"8bf86877601a471da8ebb136cc7f435f","version_major":2,"version_minor":0}

Accuracy = 0.9103999733924866
Лучше на 0.0067999958992004395

lstm_mean_last_concat_hd384_nl1_drop0.0 модель

{"model_id":"49f1daffd738466d8ddfea3f05204c08","version_major":2,"version_minor":0}

Accuracy = 0.9133999943733215
Лучше на 0.0042000412940979

lstm_mean_hd384_nl1_drop0.1 модель

{"model_id":"102f305c87d940dc8468d30e15771e7b","version_major":2,"version_minor":0}

Accuracy = 0.9151999950408936
Лучше на 0.006000041961669922

lstm_max_hd384_nl1_drop0.1 модель

{"model_id":"fd3dc38d22f24ce2bed1f1a32515be48","version_major":2,"version_minor":0}

Accuracy = 0.9121999740600586
Лучше на 0.008599996566772461

lstm_mean_last_concat_hd384_nl1_drop0.1 модель

{"model_id":"ad624ab300894fd29dcc8bfa3cc1774a","version_major":2,"version_minor":0}
```

```
Accuracy = 0.9161999821662903
Лучше на 0.00700002908706665

lstm_mean_hd384_nl1_drop0.2 модель

{"model_id":"052e5175c4f94eada0a26ce8bb99693c","version_major":2,"version_minor":0}

Accuracy = 0.9163999557495117
Лучше на 0.007200002670288086

lstm_max_hd384_nl1_drop0.2 модель

{"model_id":"34d70726d18b4752b2b678603f1de107","version_major":2,"version_minor":0}

Accuracy = 0.913599967956543
Лучше на 0.009999990463256836

lstm_mean_last_concat_hd384_nl1_drop0.2 модель

{"model_id":"d63ec7142d7e40088aac13cbbde186d4","version_major":2,"version_minor":0}

Accuracy = 0.9169999957084656
Лучше на 0.007800042629241943

lstm_mean_hd384_nl1_drop0.3 модель

{"model_id":"54e435ea86314f1d908db1ed9525b6c9","version_major":2,"version_minor":0}

Accuracy = 0.9210000038146973
Лучше на 0.011800050735473633

lstm_max_hd384_nl1_drop0.3 модель

{"model_id":"8bbf6972bdc24828b543d4e34336d21a","version_major":2,"version_minor":0}

Accuracy = 0.9149999618530273
Лучше на 0.011399984359741211

lstm_mean_last_concat_hd384_nl1_drop0.3 модель

{"model_id":"db5db5fedb69473e8e2c01698dfc999e","version_major":2,"version_minor":0}

Accuracy = 0.9099999666213989
Лучше на 0.000800013542175293

lstm_mean_hd384_nl2_drop0.0 модель
```

{"model_id":"07d719938513465aa964fd060460b987","version_major":2,"version_minor":0}

Accuracy = 0.9165999889373779
Лучше на 0.007400035858154297

lstm_max_hd384_nl2_drop0.0 модель

{"model_id":"8d6c30a9010247a59f7eebca6c53ce09","version_major":2,"version_minor":0}

Accuracy = 0.9167999625205994
Лучше на 0.013199985027313232

lstm_mean_last_concat_hd384_nl2_drop0.0 модель

{"model_id":"2c3ad4cd4ba7456a954781499b8d86c1","version_major":2,"version_minor":0}

Accuracy = 0.9210000038146973
Лучше на 0.011800050735473633

lstm_mean_hd384_nl2_drop0.1 модель

{"model_id":"2dfb1fe9b896488dabeb833b96a05efb","version_major":2,"version_minor":0}

Accuracy = 0.9165999889373779
Лучше на 0.007400035858154297

lstm_max_hd384_nl2_drop0.1 модель

{"model_id":"dc09dcc9452e45c08902fddc1f4b4c54","version_major":2,"version_minor":0}

Accuracy = 0.9174000024795532
Лучше на 0.01380002498626709

lstm_mean_last_concat_hd384_nl2_drop0.1 модель

{"model_id":"e6f29a97610e48c7980e24b4b9d9757a","version_major":2,"version_minor":0}

Accuracy = 0.9179999828338623
Лучше на 0.008800029754638672

lstm_mean_hd384_nl2_drop0.2 модель

{"model_id":"0d23988fdf704cf4aa7e1baa9dd4f180","version_major":2,"version_minor":0}

Accuracy = 0.91839998960495
Лучше на 0.009200036525726318

```
lstm_max_hd384_nl2_drop0.2 модель

{"model_id":"cf2b3f0160d4457e945c793ae4bc3b0d","version_major":2,"version_minor":0}

Accuracy = 0.9187999963760376
Лучше на 0.015200018882751465

lstm_mean_last_concat_hd384_nl2_drop0.2 модель

{"model_id":"e5ea3a1527b4493e8edc6cf0ba805476","version_major":2,"version_minor":0}

Accuracy = 0.915399968624115
Лучше на 0.006200015544891357

lstm_mean_hd384_nl2_drop0.3 модель

{"model_id":"93f7443fbc944219833c9ad3927ab9a0","version_major":2,"version_minor":0}

Accuracy = 0.91839998960495
Лучше на 0.009200036525726318

lstm_max_hd384_nl2_drop0.3 модель

{"model_id":"4e13663218034de6aeaf4cc2e3314596","version_major":2,"version_minor":0}

Accuracy = 0.9147999882698059
Лучше на 0.011200010776519775

lstm_mean_last_concat_hd384_nl2_drop0.3 модель

{"model_id":"c48a1cc2cc654ec1a098d29b8ba44dba","version_major":2,"version_minor":0}

Accuracy = 0.9187999963760376
Лучше на 0.009600043296813965

lstm_mean_hd384_nl3_drop0.0 модель

{"model_id":"073ed3d67af3494581fa81aab735b15d","version_major":2,"version_minor":0}

Accuracy = 0.9139999747276306
Лучше на 0.004800021648406982

lstm_max_hd384_nl3_drop0.0 модель

{"model_id":"1cedeb6be3ca4fb2a6f86d867af990f6","version_major":2,"version_minor":0}
```

```
Accuracy = 0.915399968624115
Лучше на 0.011799991130828857

lstm_mean_last_concat_hd384_nl3_drop0.0 модель

{"model_id":"43e4177ffb6643a589c66f07bbfc87f1","version_major":2,"version_minor":0}

Accuracy = 0.9169999957084656
Лучше на 0.007800042629241943

lstm_mean_hd384_nl3_drop0.1 модель

{"model_id":"319b4688656f4861bab9b5f538d8764f","version_major":2,"version_minor":0}

Accuracy = 0.9195999503135681
Лучше на 0.010399997234344482

lstm_max_hd384_nl3_drop0.1 модель

{"model_id":"2f30b4534ee74a299e5d51a6bc596a68","version_major":2,"version_minor":0}

Accuracy = 0.9163999557495117
Лучше на 0.012799978256225586

lstm_mean_last_concat_hd384_nl3_drop0.1 модель

{"model_id":"cd72d853297648a0b2f7b50d501b9326","version_major":2,"version_minor":0}

Accuracy = 0.9187999963760376
Лучше на 0.009600043296813965

lstm_mean_hd384_nl3_drop0.2 модель

{"model_id":"b41ce9c7156842799a87d91be0127104","version_major":2,"version_minor":0}

Accuracy = 0.9205999970436096
Лучше на 0.011400043964385986

lstm_max_hd384_nl3_drop0.2 модель

{"model_id":"16d36796e423426dba60b0d37e36363f","version_major":2,"version_minor":0}

Accuracy = 0.9175999760627747
Лучше на 0.013999998569488525

lstm_mean_last_concat_hd384_nl3_drop0.2 модель
```

{"model_id":"f3b8acff34ab49489b632ebe2d2c9b47","version_major":2,"version_minor":0}

Accuracy = 0.9159999489784241
Лучше на 0.0067999958992004395

lstm_mean_hd384_nl3_drop0.3 модель

{"model_id":"436dda410cd840e2be6ecc5bf6532d37","version_major":2,"version_minor":0}

Accuracy = 0.9161999821662903
Лучше на 0.00700002908706665

lstm_max_hd384_nl3_drop0.3 модель

{"model_id":"e1e5da0218194e75a94b6c7e79d234cf","version_major":2,"version_minor":0}

Accuracy = 0.918999969959259
Лучше на 0.0153999924659729

lstm_mean_last_concat_hd384_nl3_drop0.3 модель

{"model_id":"c37f3f542b7d4a2984c955d2e27da75a","version_major":2,"version_minor":0}

Accuracy = 0.9181999564170837
Лучше на 0.009000003337860107

lstm_mean_hd512_nl1_drop0.0 модель

{"model_id":"8c6bf666c5404c84978acc1fd710fe13","version_major":2,"version_minor":0}

Accuracy = 0.920799970626831
Лучше на 0.011600017547607422

lstm_max_hd512_nl1_drop0.0 модель

{"model_id":"68700cd072af4340aac04b67d864c666","version_major":2,"version_minor":0}

Accuracy = 0.9103999733924866
Лучше на 0.0067999958992004395

lstm_mean_last_concat_hd512_nl1_drop0.0 модель

{"model_id":"b7b10f782b6b4f399a0399e7e120ab1e","version_major":2,"version_minor":0}

Accuracy = 0.9177999496459961
Лучше на 0.008599996566772461

```
lstm_mean_hd512_nl1_drop0.1 модель

{"model_id":"b7d4eb54465b45a9a6d8c5318edab6e3","version_major":2,"vers
ion_minor":0}

Accuracy = 0.9179999828338623
Лучше на 0.008800029754638672

lstm_max_hd512_nl1_drop0.1 модель

{"model_id":"804ed6482eb541f6b6570fd2cfd34299","version_major":2,"vers
ion_minor":0}

Accuracy = 0.9129999876022339
Лучше на 0.009400010108947754

lstm_mean_last_concat_hd512_nl1_drop0.1 модель

{"model_id":"1424ece4e72545d484b0ba544d80617c","version_major":2,"vers
ion_minor":0}

Accuracy = 0.9151999950408936
Лучше на 0.006000041961669922

lstm_mean_hd512_nl1_drop0.2 модель

{"model_id":"694aaccf35b14568b9efe3c84e7dabf5","version_major":2,"vers
ion_minor":0}

Accuracy = 0.9177999496459961
Лучше на 0.008599996566772461

lstm_max_hd512_nl1_drop0.2 модель

{"model_id":"8fb64dede50f4b6e8ae50cc5195d2122","version_major":2,"vers
ion_minor":0}

Accuracy = 0.915399968624115
Лучше на 0.011799991130828857

lstm_mean_last_concat_hd512_nl1_drop0.2 модель

{"model_id":"a8237899da124a2f83ea67e9741cbb47","version_major":2,"vers
ion_minor":0}

Accuracy = 0.9159999489784241
Лучше на 0.006799995899200439 5

lstm_mean_hd512_nl1_drop0.3 модель

{"model_id":"ade0b71e98064991a6cd5c6a35d6eeac","version_major":2,"vers
ion_minor":0}
```

```
Accuracy = 0.9221999645233154
Лучше на 0.013000011444091797

lstm_max_hd512_nl1_drop0.3 модель

{"model_id":"86b17a38f95f4936924bc482b91f8535","version_major":2,"version_minor":0}

Accuracy = 0.9145999550819397
Лучше на 0.010999977588653564

lstm_mean_last_concat_hd512_nl1_drop0.3 модель

{"model_id":"1cd5c0359be04b129eda9da6685b7d7f","version_major":2,"version_minor":0}

Accuracy = 0.9147999882698059
Лучше на 0.005600035190582275

lstm_mean_hd512_nl2_drop0.0 модель

{"model_id":"bcbd4adbea04450b8e272ae705daf29e","version_major":2,"version_minor":0}

Accuracy = 0.9175999760627747
Лучше на 0.008400022983551025

lstm_max_hd512_nl2_drop0.0 модель

{"model_id":"512c6854fe454b1cb1aa6f09aec75ce8","version_major":2,"version_minor":0}

Accuracy = 0.9177999496459961
Лучше на 0.014199972152709961

lstm_mean_last_concat_hd512_nl2_drop0.0 модель

{"model_id":"fd1c0a3041aa46e8b89b4baf6774787f","version_major":2,"version_minor":0}

Accuracy = 0.9129999876022339
Лучше на 0.003800034523010254

lstm_mean_hd512_nl2_drop0.1 модель

{"model_id":"35c7d73829ad4fbcac236635d233ae03","version_major":2,"version_minor":0}

Accuracy = 0.91839998960495
Лучше на 0.009200036525726318

lstm_max_hd512_nl2_drop0.1 модель
```

{"model_id":"34e046c801fd4c26984767321414fc6f","version_major":2,"version_minor":0}

Accuracy = 0.9145999550819397
Лучше на 0.010999977588653564

lstm_mean_last_concat_hd512_nl2_drop0.1 модель

{"model_id":"90ca64ddedab4b77a2931d7934a339e4","version_major":2,"version_minor":0}

Accuracy = 0.9179999828338623
Лучше на 0.008800029754638672

lstm_mean_hd512_nl2_drop0.2 модель

{"model_id":"a6ebcaae91e144c5a809edd81789248b","version_major":2,"version_minor":0}

Accuracy = 0.9156000018119812
Лучше на 0.006400048732757568

lstm_max_hd512_nl2_drop0.2 модель

{"model_id":"61bc5740463e4d81a49db2cf3438ac46","version_major":2,"version_minor":0}

Accuracy = 0.9165999889373779
Лучше на 0.013000011444091797

lstm_mean_last_concat_hd512_nl2_drop0.2 модель

{"model_id":"d6e97f2583ab4669b66f512247115ba6","version_major":2,"version_minor":0}

Accuracy = 0.915399968624115
Лучше на 0.006200015544891357

lstm_mean_hd512_nl2_drop0.3 модель

{"model_id":"4b1669bd5b4946f881d19e5542b53577","version_major":2,"version_minor":0}

Accuracy = 0.9174000024795532
Лучше на 0.00820004940032959

lstm_max_hd512_nl2_drop0.3 модель

{"model_id":"72eb41ca03b24bd282d59f7153f9cb2f","version_major":2,"version_minor":0}

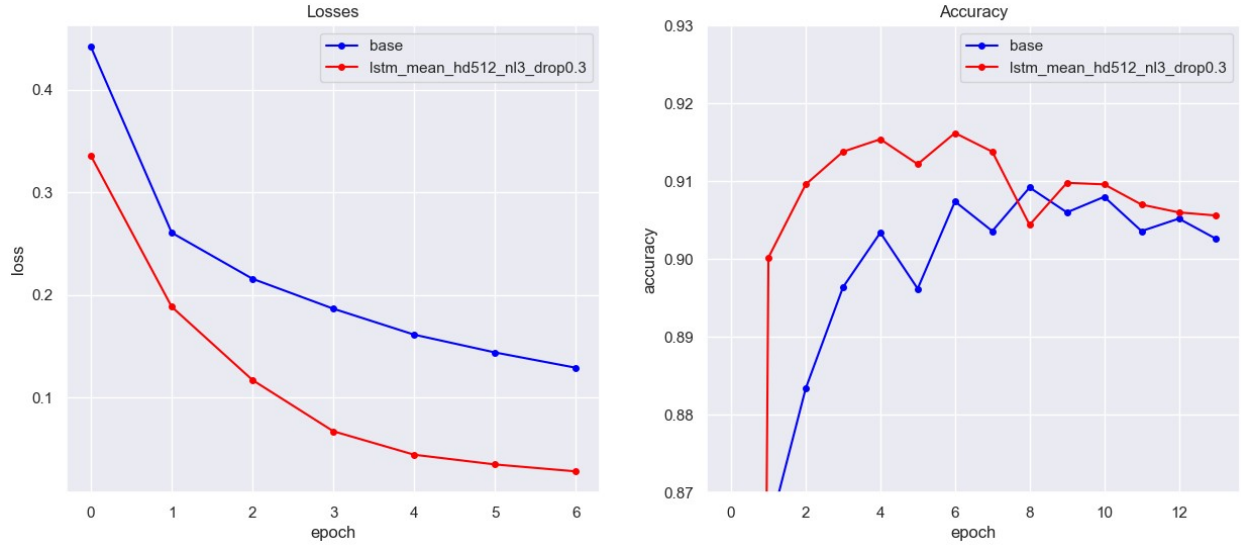
Accuracy = 0.9193999767303467
Лучше на 0.015799999237060547

```
lstm_mean_last_concat_hd512_nl2_drop0.3 модель

{"model_id":"a9fe12acbc1e4a8385aa0de9c13a7c09","version_major":2,"version_minor":0}

Accuracy = 0.9195999503135681
Лучше на 0.010399997234344482

lstm_mean_hd512_nl3_drop0.0 модель

{"model_id":"8e4839e768ae463d8d343ff8c4707da5","version_major":2,"version_minor":0}

Accuracy = 0.9181999564170837
Лучше на 0.009000003337860107

lstm_max_hd512_nl3_drop0.0 модель

{"model_id":"fecd6d551b434fad912576ee54b3aeda","version_major":2,"version_minor":0}

Accuracy = 0.920199990272522
Лучше на 0.01660001277923584

lstm_mean_last_concat_hd512_nl3_drop0.0 модель

{"model_id":"3c5be73b6dbf4a039dee6bd209b02054","version_major":2,"version_minor":0}

Accuracy = 0.9156000018119812
Лучше на 0.006400048732757568

lstm_mean_hd512_nl3_drop0.1 модель

{"model_id":"2e569508dd5446539ba2400d6e106b92","version_major":2,"version_minor":0}

Accuracy = 0.9192000031471252
Лучше на 0.010000050067901611

lstm_max_hd512_nl3_drop0.1 модель

{"model_id":"1c0801c062b34f449d2ad114cb55f66d","version_major":2,"version_minor":0}

Accuracy = 0.9121999740600586
Лучше на 0.008599996566772461

lstm_mean_last_concat_hd512_nl3_drop0.1 модель

{"model_id":"77c53c04ed2240acad79f236f57d2385","version_major":2,"version_minor":0}
```

```
Accuracy = 0.91839998960495
Лучше на 0.009200036525726318

lstm_mean_hd512_nl3_drop0.2 модель

{"model_id":"3ccaae22045b4acc84d2e69a94717d38","version_major":2,"version_minor":0}

Accuracy = 0.9179999828338623
Лучше на 0.008800029754638672

lstm_max_hd512_nl3_drop0.2 модель

{"model_id":"a7db92b76bb14b598cfd4c3db2fe71b5","version_major":2,"version_minor":0}

Accuracy = 0.9149999618530273
Лучше на 0.011399984359741211

lstm_mean_last_concat_hd512_nl3_drop0.2 модель

{"model_id":"2e72479068894eacb1d27be2430d3bc6","version_major":2,"version_minor":0}

Accuracy = 0.9179999828338623
Лучше на 0.008800029754638672

lstm_mean_hd512_nl3_drop0.3 модель

{"model_id":"1728a2eebf354fa19a533324cae5ac42","version_major":2,"version_minor":0}

Accuracy = 0.9167999625205994
Лучше на 0.007600009441375732

lstm_max_hd512_nl3_drop0.3 модель

{"model_id":"80d2c3e0fe564246af33ee7878dada99","version_major":2,"version_minor":0}

Accuracy = 0.9179999828338623
Лучше на 0.014400005340576172

lstm_mean_last_concat_hd512_nl3_drop0.3 модель

{"model_id":"c4cbbbaf3b5441a8858976c1ed4f54a9","version_major":2,"version_minor":0}

Accuracy = 0.9177999496459961
Лучше на 0.008599996566772461

print(f'Лучшая accuracy: {best_acc}')
print(f'Лучшие параметры: {best_params}')
```

```
Лучшая accuracy: 0.9221999645233154
Лучшие параметры: {'hidden_dim': 512, 'aggregation_type': 'mean',
'num_layers': 1, 'dropout_rate': 0.3}
```

```
num_epoch = 7

model = LSTMCharLM(**best_params, vocab_size=len(vocab)).to(device)
criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
optimizer = torch.optim.Adam(model.parameters())

agg_type = best_params['aggregation_type']

model_loss, model_acc = train_model(model, criterion, optimizer,
model_name=f'{name_model}_{agg_type}_hd{hidden_dim}_nl{num_layers}_dro
p{dropout_rate}',
                                    aggregation_type=agg_type,
plot=True, num_epoch=num_epoch, base_mean_loss=base_mean_loss,
base_mean_acc=base_mean_acc)
```

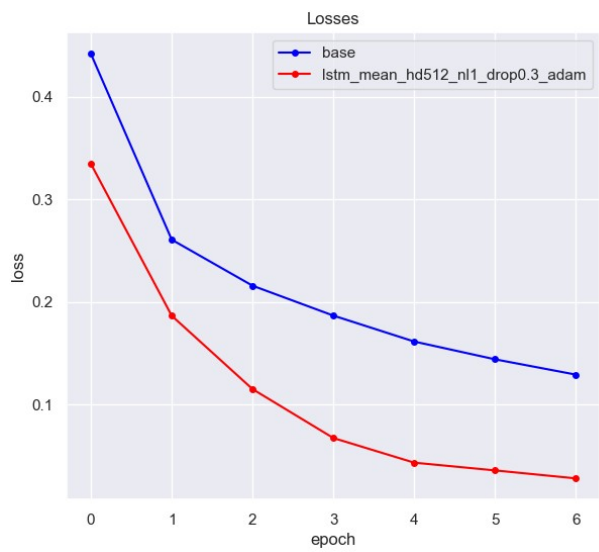


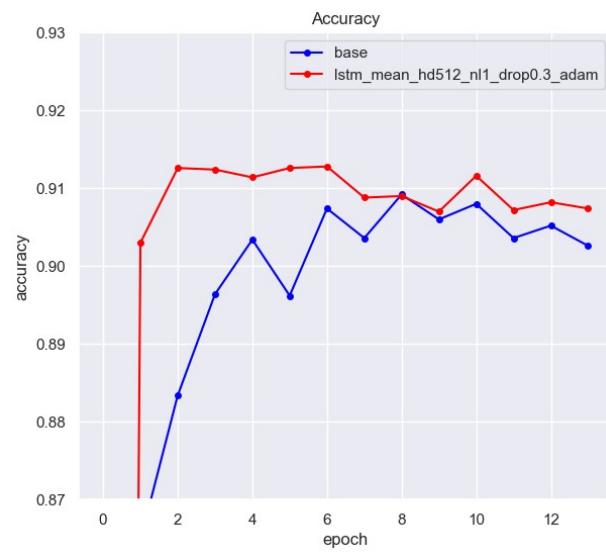

```
Accuracy = 0.9161999821662903
Лучше на 0.00700002908706665
```

Выбор оптимизатора

```
['adam', 'adamW_wd01', 'adamW_wd03', 'adamW_wd05']

num_epoch = 7

best_params = {'hidden_dim': 512, 'aggregation_type': 'mean',
'num_layers': 1, 'dropout_rate': 0.3}
agg_type = best_params['aggregation_type']
name = 'adam'
```

```
model = LSTMCharLM(**best_params, vocab_size=len(vocab)).to(device)
criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
optimizer = torch.optim.Adam(model.parameters())

model_loss, model_acc = train_model(model, criterion, optimizer,

model_name=f'{name_model}_{agg_type}_hd{best_params["hidden_dim"]}_nl{
best_params["num_layers"]}_drop{best_params["dropout_rate"]}_{name}',
                                    aggregation_type=agg_type,
plot=True, num_epoch=num_epoch, base_mean_loss=base_mean_loss,
base_mean_acc=base_mean_acc)

curr_acc = max(model_acc)
if curr_acc > best_acc:
    best_acc = curr_acc
    best_optimizer_name = name
    best_optimizer = optimizer
```



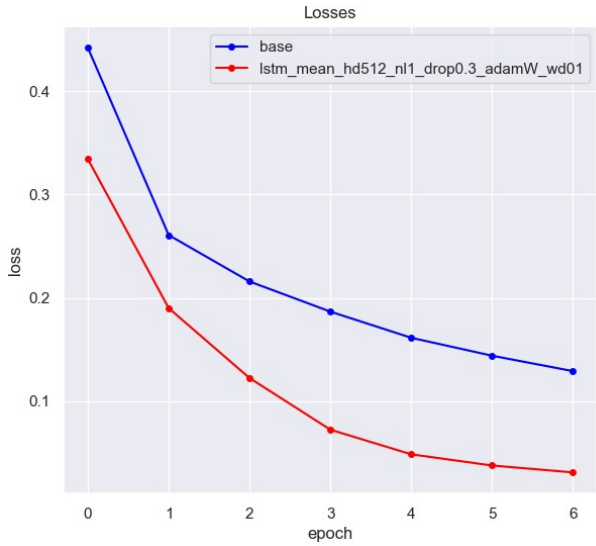


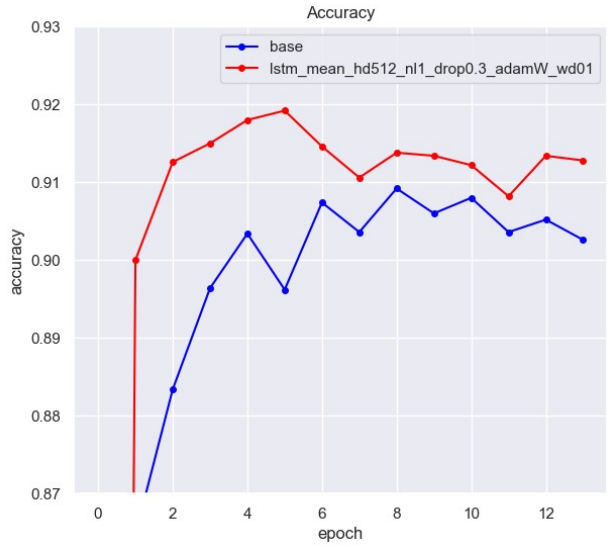

```
Accuracy = 0.9127999544143677
Лучше на 0.003600001335144043
```

Adam заменим на AdamW и попробуем подобрать для него наилучшие параметры

```
['adam', 'adamW_wd01', 'adamW_wd03', 'adamW_wd05']

num_epoch = 7

best_params = {'hidden_dim': 512, 'aggregation_type': 'mean',
'num_layers': 1, 'dropout_rate': 0.3}
agg_type = best_params['aggregation_type']
name = 'adamW_wd01'
```

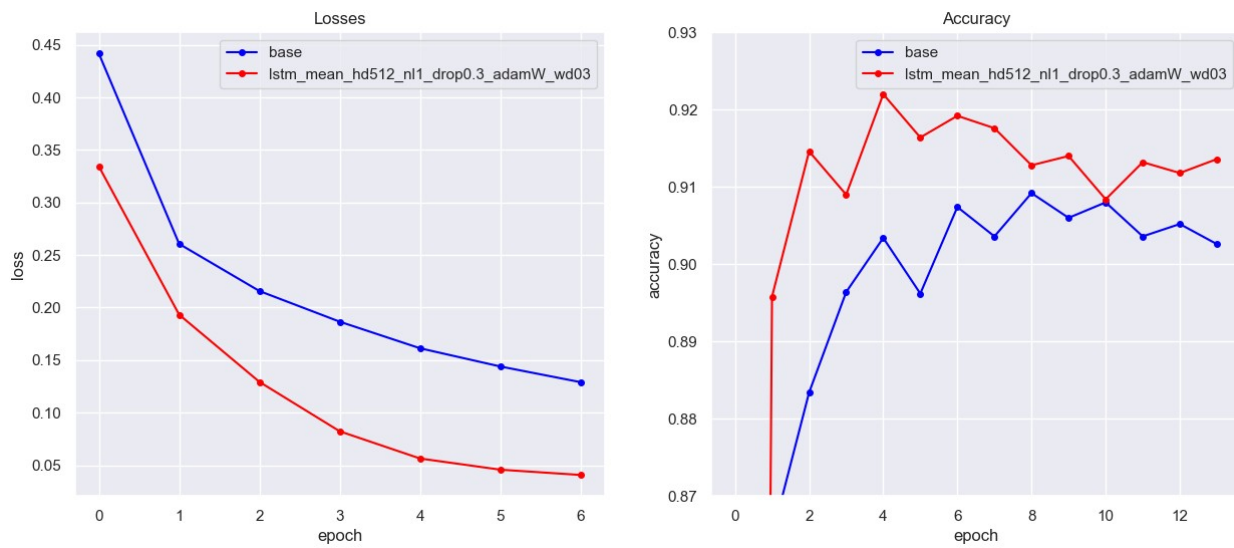
```
model = LSTMCharLM(**best_params, vocab_size=len(vocab)).to(device)
criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
optimizer = torch.optim.AdamW(model.parameters(), lr=0.001,
weight_decay=0.01)

model_loss, model_acc = train_model(model, criterion, optimizer,

model_name=f'{name_model}_{agg_type}_hd{best_params["hidden_dim"]}_nl{
best_params["num_layers"]}_drop{best_params["dropout_rate"]}_{name}',
                                    aggregation_type=agg_type,
plot=True, num_epoch=num_epoch, base_mean_loss=base_mean_loss,
base_mean_acc=base_mean_acc)

curr_acc = max(model_acc)
if curr_acc > best_acc:
    best_acc = curr_acc
    best_optimizer_name = name
    best_optimizer = optimizer
```





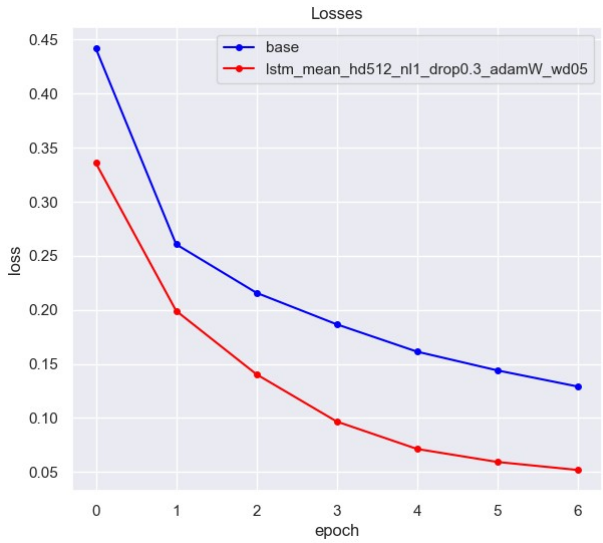

```
Accuracy = 0.9192000031471252
Лучше на 0.010000050067901611

['adam', 'adamW_wd01', 'adamW_wd03', 'adamW_wd05']

num_epoch = 7

best_params = {'hidden_dim': 512, 'aggregation_type': 'mean',
'num_layers': 1, 'dropout_rate': 0.3}
agg_type = best_params['aggregation_type']
name = 'adamW_wd03'

model = LSTMCharLM(**best_params, vocab_size=len(vocab)).to(device)
criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
```

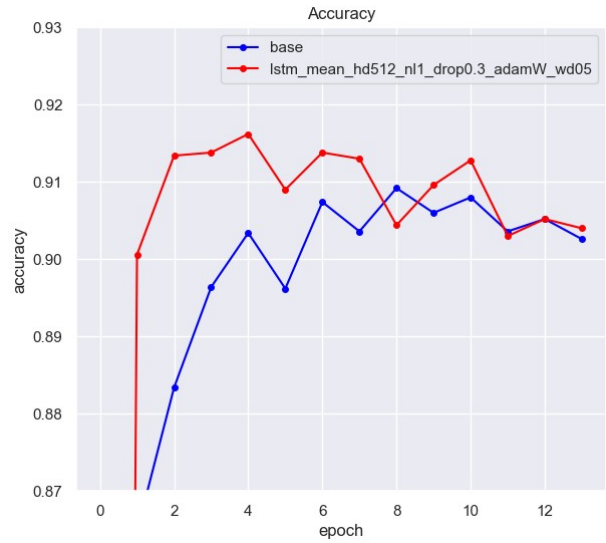
```
optimizer = torch.optim.AdamW(model.parameters(), lr=0.001,
weight_decay=0.03)

model_loss, model_acc = train_model(model, criterion, optimizer,

model_name=f'{name_model}_{agg_type}_hd{best_params["hidden_dim"]}_nl{
best_params["num_layers"]}_drop{best_params["dropout_rate"]}_{name}',
                                    aggregation_type=agg_type,
plot=True, num_epoch=num_epoch, base_mean_loss=base_mean_loss,
base_mean_acc=base_mean_acc)

curr_acc = max(model_acc)
if curr_acc > best_acc:
    best_acc = curr_acc
    best_optimizer_name = name
    best_optimizer = optimizer
```




```
Accuracy = 0.921999990940094
Лучше на 0.012800037860870361

['adam', 'adamW_wd01', 'adamW_wd03', 'adamW_wd05']

num_epoch = 7

best_params = {'hidden_dim': 512, 'aggregation_type': 'mean',
'num_layers': 1, 'dropout_rate': 0.3}
agg_type = best_params['aggregation_type']
name = 'adamW_wd05'

model = LSTMCharLM(**best_params, vocab_size=len(vocab)).to(device)
criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
optimizer = torch.optim.AdamW(model.parameters(), lr=0.001,
weight_decay=0.05)
```

```
model_loss, model_acc = train_model(model, criterion, optimizer,

model_name=f'{name_model}_{agg_type}_hd{best_params["hidden_dim"]}_nl{
best_params["num_layers"]}_drop{best_params["dropout_rate"]}_{name}',
                                    aggregation_type=agg_type,
plot=True, num_epoch=num_epoch, base_mean_loss=base_mean_loss,
base_mean_acc=base_mean_acc)

curr_acc = max(model_acc)
if curr_acc > best_acc:
    best_acc = curr_acc
    best_optimizer_name = name
    best_optimizer = optimizer
```






```
Accuracy = 0.9161999821662903
Лучше на 0.00700002908706665

print(f'Лучшая accuracy: {best_acc}')
print(f'Лучший оптимайзер: {best_optimizer_name}')

Лучшая accuracy: 0.921999990940094
Лучший оптимайзер: adamW_wd03
```

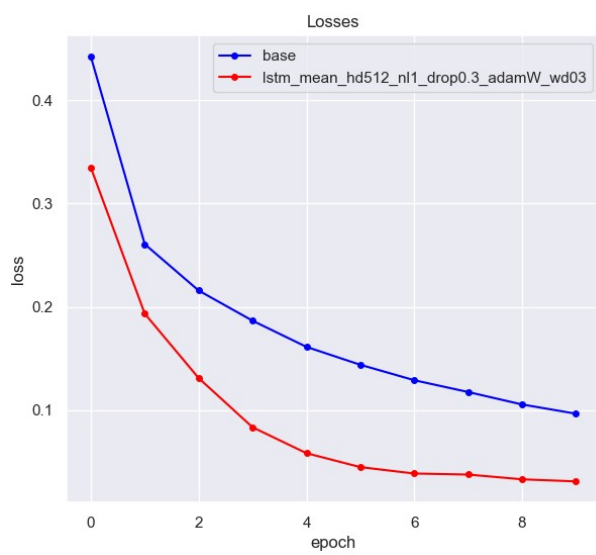
Получение высокого качества (3 балла)

В конце вашей работы вы должны указать, какая из моделей дала лучший результат, и вывести качество, которое дает лучшая модель, с помощью функции `evaluate`. Ваша модель будет оцениваться по метрике `accuracy` следующим образом:

- $accuracy < 0.9$ --- 0 баллов;
- $0.9 \leq accuracy < 0.91$ --- 1 балл;

- $0.91 \leq accuracy < 0.915$ --- 2 балла;
- $0.915 \leq accuracy$ --- 3 балла.

```
name_model = 'lstm'
best_params = {'hidden_dim': 512, 'aggregation_type': 'mean',
'num_layers': 1, 'dropout_rate': 0.3}
agg_type = best_params['aggregation_type']
name = 'adamW_wd03'

num_epoch = 10

model = LSTMCharLM(**best_params, vocab_size=len(vocab)).to(device)
criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
optimizer = torch.optim.AdamW(model.parameters(), lr=0.001,
weight_decay=0.03)

model_loss, model_acc = train_model(model, criterion, optimizer,

model_name=f'{name_model}_{agg_type}_hd{best_params["hidden_dim"]}_nl{
best_params["num_layers"]}_drop{best_params["dropout_rate"]}_{name}',
                                        aggregation_type=agg_type,
plot=True, num_epoch=num_epoch, base_mean_loss=base_mean_loss,
base_mean_acc=base_mean_acc)
```



```
Accuracy = 0.9211999773979187
Лучше на 0.012000024318695068

print(f'Лучшая accuracy: {max(model_acc)}')
print(f'{name_model}_{agg_type}_hd{best_params["hidden_dim"]}_nl{best_
params["num_layers"]}_drop{best_params["dropout_rate"]}_{name}')

Лучшая accuracy: 0.9211999773979187
lstm_mean_hd512_nl1_drop0.3_adamW_wd03
```

Оформление отчета (2 балла)

В конце работы подробно опишите все проведенные эксперименты.

- Укажите, какие из экспериментов принесли улучшение, а какие --- нет.
- Проанализируйте графики сходимости моделей в проведенных экспериментах. Являются ли колебания качества обученных моделей существенными в зависимости от эпохи обучения, или же сходимость стабильная?
- Укажите, какая модель получилась оптимальной.
- Было опробовано: -- Агрегации ['max', 'mean', 'sum', 'max_norm', 'mean_norm', 'mean_last_concat', 'multi_concat']. Хорошо себя показали агрегации: 'max', 'mean', 'mean_last_concat', 'multi_concat. -- Изменение num_layers и dropout_rate. Прирост точности был при num_layers=2, но был незначительный. -- Модели LSTM, GRU и Bidirectional RNN. Все они показали точность выше, чем Base model. Выбрал LSTM. -- Подбор гиперпараметров для модели LSTM, количество эпох было увеличено для лучшей сходимости. Подбор осуществлялся по гиперпараметрам с дальнейшем выбором сохранением лучшего набор: hidden_dims = [256, 384, 512] aggregation_types = ['mean', 'max', 'mean_last_concat'] num_layers_list = [1, 2, 3] dropout_rates = [0.0, 0.1, 0.2, 0.3]. -- Выбор оптимизатора Adam, AdamW с различными параметрами регуляризации.
- Лосс при проведении экспериментов стабильно уменьшался, выходил на плато примерно к 5 эпохе. Accuracy во всех моделях на валидационном датасете выходит на плато с к 3-5 эпохе с незначительными флуктуациями (чуть позже при увеличении слоев).
- Модель LSTM с подобранными гиперпараметрами, агрегацией и оптимизатором ('hidden_dim': 512, 'aggregation_type': 'mean', 'num_layers': 1, 'dropout_rate': 0.3, AdamW(lr=0.001, weight_decay=0.03)) показала лучший результат, который составил Accuracy ~ 0.92.

```
df_stats_sorted_acc = df_stats.sort_values('Max_accuracy',
ascending=False, ignore_index=True).copy()
display(df_stats_sorted_acc.head(10))
```

```
                                     Model  Min_loss  Max_accuracy  \
0              lstm_mean_hd512_nl1_drop0.3  0.029084        0.9222
1  lstm_mean_last_concat_hd384_nl2_drop0.0  0.029686        0.9210
2              lstm_mean_hd384_nl1_drop0.3  0.026763        0.9210
3              lstm_mean_hd512_nl1_drop0.0  0.028680        0.9208
4              lstm_mean_hd384_nl3_drop0.2  0.043459        0.9206
5              lstm_mean_hd256_nl3_drop0.2  0.046855        0.9204
6               lstm_max_hd512_nl3_drop0.0  0.043328        0.9202
7              lstm_mean_hd384_nl3_drop0.1  0.039178        0.9196
8  lstm_mean_last_concat_hd512_nl2_drop0.3  0.039498        0.9196
9               lstm_max_hd512_nl2_drop0.3  0.055904        0.9194

   Diff_accuracy
0         0.0130
1         0.0118
2         0.0118
```

```
3          0.0116
4          0.0114
5          0.0112
6          0.0166
7          0.0104
8          0.0104
9          0.0158
```

```
# df_stats_sorted_acc.to_csv('hw_2_model_stats.csv', index=False)
```